

O'REILLY®

Second Edition

Head First

JavaScript Programming

A Learner's Guide to
Modern JavaScript

Eric Freeman &
Elisabeth Robson

Early
Release

RAW &
UNEDITED



A Brain-Friendly Guide

Head First JavaScript Programming

SECOND EDITION

A Learner's Guide to Modern JavaScript

Eric Freeman and Elisabeth Robson



Beijing • Boston • Farnham • Sebastopol • Tokyo

OceanofPDF.com

Head First JavaScript Programming

by Eric Freeman and Elisabeth Robson

Copyright © 2025 Eric Freeman and Elisabeth Robson. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisitions Editor: David Michelson
- Development Editor: Michele Cronin
- Production Editor: Christopher Faucher
- Interior Designer: David Futato
- Cover Designer: Susan Thompson, based on a series design by Ellie Volckhausen
- Illustrator: José Marzan Jr.

- March 2025: Second Edition

Revision History for the Second Edition

- 2024-02-06: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098147945> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Head First JavaScript Programming*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14788-4

OceanofPDF.com

Brief Table of Contents (*Not Yet Final*)

Chapter 1, A Quick Dip into JavaScript: Getting Your Feet Wet (available)

Chapter 2, Writing Real Code: Going Further (available)

Chapter 3, Introducing Functions: Getting Functional (unavailable)

Chapter 4, Putting Some Order in Your Data: Arrays (unavailable)

Chapter 5, Understanding Objects: A Trip to Objectville (unavailable)

Chapter 6, Interacting With Your Web Page: Getting to Know the DOM
(unavailable)

Chapter 7, Types, Equality, Conversion, and All that Jazz: Serious Types
(unavailable)

Chapter 8, Bringing It All Together: Building an App (unavailable)

Chapter 9, Asynchronous Coding: Handling Events (unavailable)

Chapter 10, First Class Functions: Liberated Functions (unavailable)

Chapter 11, Anonymous Functions, Scope, and Closures: Serious Functions
(unavailable)

Chapter 12, Advanced Object Construction: Creating Objects (unavailable)

Chapter 13, Using Prototypes: Extra Strength Objects (unavailable)

Appendix A, Leftovers: The Top Ten Topics (we didn't cover) (unavailable)

OceanofPDF.com

Chapter 1. A Quick Dip into Javascript: Getting your Feet Wet

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at mcronin@oreilly.com.

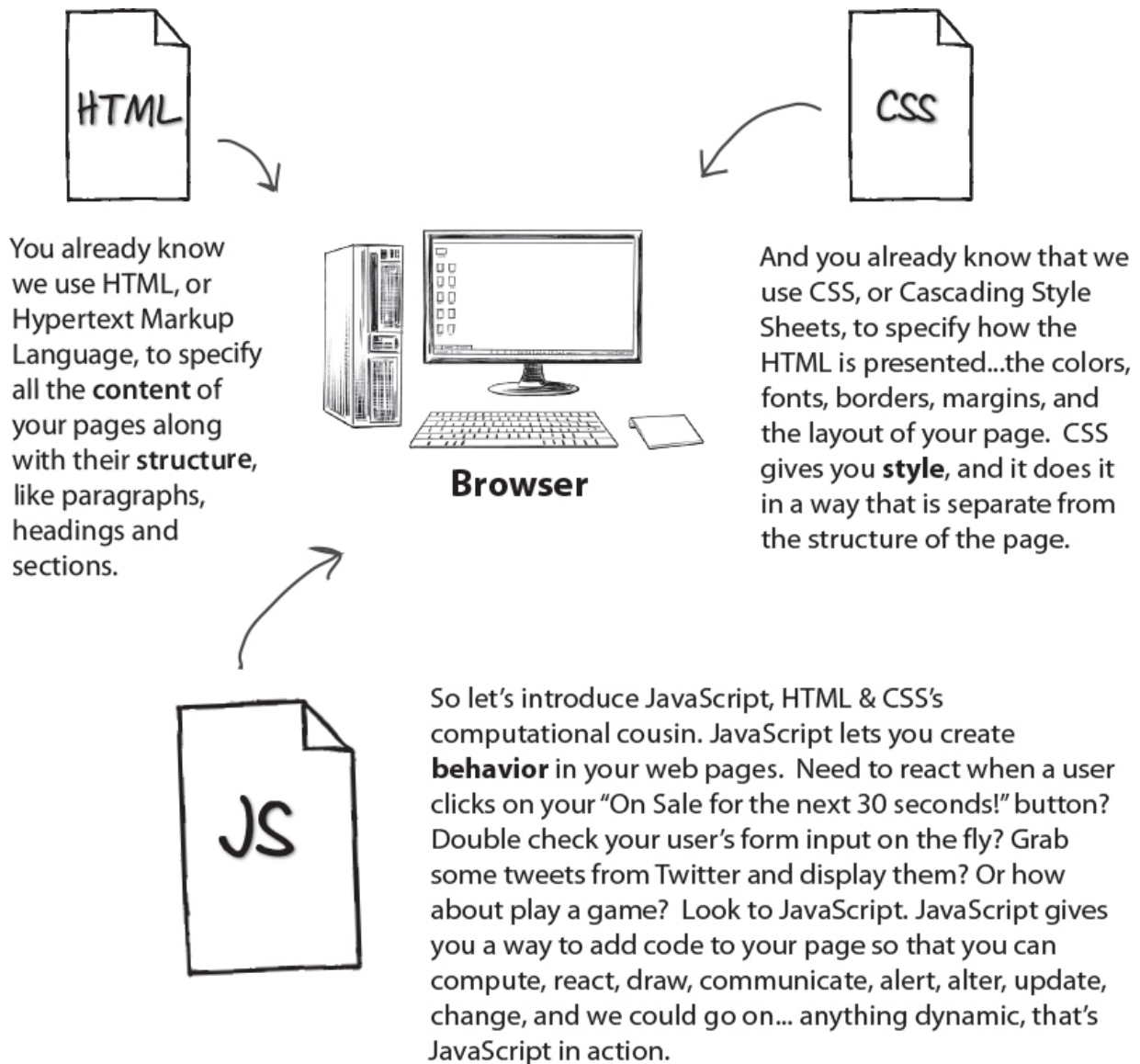


JavaScript gives you superpowers. The **true programming language** of the web, JavaScript lets you **add behavior** to your web pages. No more dry, boring, static pages that just sit there looking at you—with JavaScript, you'll be able to reach out and touch your users, react to interesting events, grab data from the web to use in your pages, draw graphics right in your web pages and a lot more. And once you know JavaScript you'll also be in a position to create **totally new** behaviors for your users.

You'll be in good company too. JavaScript's not only one of the **most popular** programming languages, it's also **supported** in all modern browsers and is used in many environments outside of the browser. More on that later; for now, let's get started!

The way JavaScript works

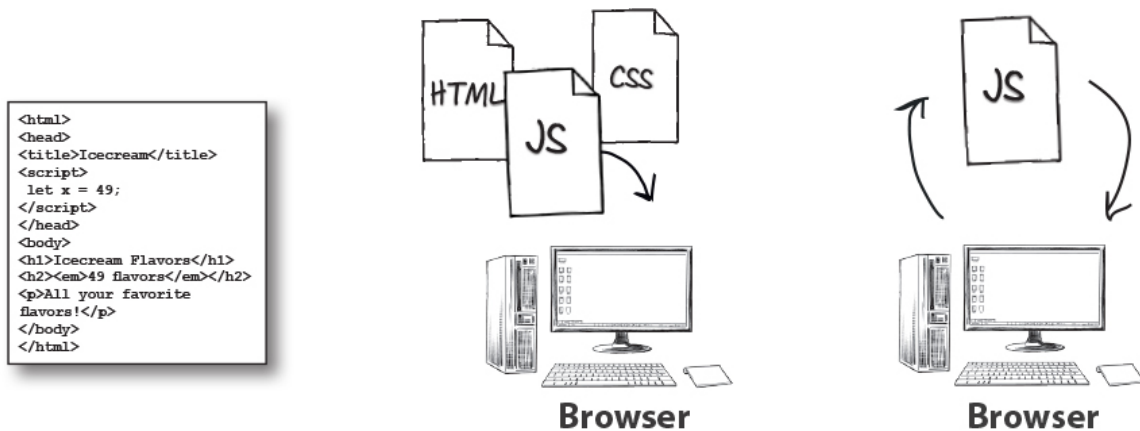
If you're used to creating structure, content, layout and style in your web pages, isn't it time you add a little behavior as well? After all, there's no need for the page to just *sit there*. Great pages should be *interactive* and *dynamic*. That's where JavaScript comes in. Let's start by taking a look at how JavaScript fits into the *web page ecosystem*:



How you're going to write JavaScript

JavaScript is fairly unique in the programming world. With your typical programming language you have to write it, compile it, link it and deploy it. JavaScript is much more fluid and flexible. With JavaScript all you need to do is write JavaScript right into your page, and then load it into a browser.

From there, the browser will happily begin executing your code. Let's take a closer look at how this works:



1 Writing

You create your page just like you always do, with HTML content and CSS style. And you also include JavaScript in your page. As you'll see, just like HTML and CSS, you can put everything together in one file, or you can place JavaScript in its own file, to be included in your page.

↑ We'll talk about the best way in a bit...

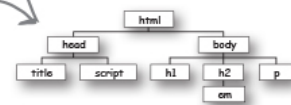
2 Loading

Point your browser to your page, just as you always do. The browser sees the code and begins parsing it immediately, getting ready to execute it. Note that like HTML and CSS, if the browser sees errors in your code, it will do its best to keep moving and reading more JavaScript, HTML and CSS. The last thing it wants to do is not be able to give the user a page to see.

For future reference, the browser also builds an "object model" of your HTML page that JavaScript can make use of. Put that in the back of your brain, we'll come back to it later...

3 Executing

The browser starts executing your code as soon as it encounters the code in your page, and continues executing it for the lifetime of your page. Unlike early versions of JavaScript, today's JavaScript is a powerhouse, using advanced compilation techniques to execute your code at nearly the same speed as many native programming languages.



How to get JavaScript into your page

First things first. You can't get very far with JavaScript if you don't know how to get it into a web page. So, how do you do that? Using the `<script>` element of course!

Let's take a boring old, garden-variety web page and add some dynamic behavior using a `<script>` element. Now, at this point, don't worry too much about the details of what we're putting into the `<script>` element—your goal right now is to get some JavaScript working.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Just a Generic Page</title>
    <script>
      setTimeout(wakeUpUser, 5000);
      function wakeUpUser() {
        alert("Are you going to stare at this boring page forever?");
      }
    </script>
  </head>
  <body>
    <h1>Just a generic heading</h1>
    <p>Not a lot to read about here. I'm just an obligatory paragraph living in
an example in a JavaScript book. I'm looking for something to make my life more
exciting.</p>
  </body>
</html>
```

Here's our standard HTML doctype, and `<html>` and `<head>` elements.

And we've got a pretty generic `<body>` for this page as well.

Ah, but we've added a script element to the `<head>` of the page.

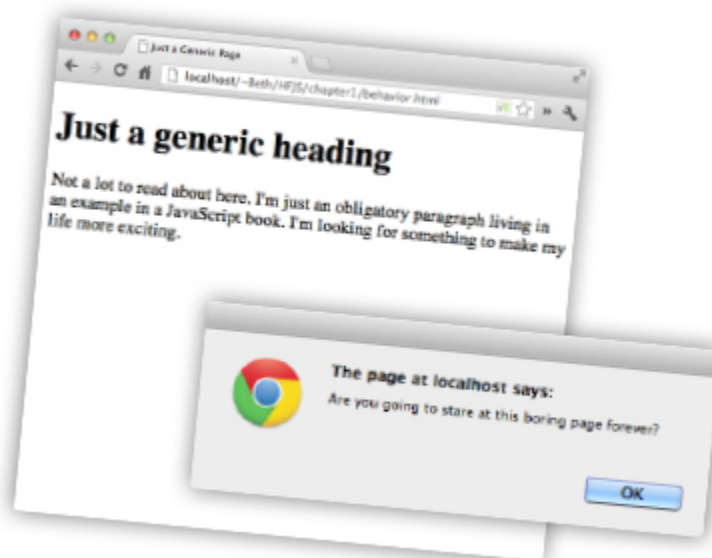
And we've written some JavaScript code inside it.

Again, don't worry too much about what this code does. Then again, we bet you'll want to take a look at the code and see if you can think through what each part might do.

A little test drive



Go ahead and type this page into a file named “behavior.html”. Drag the file to your browser (or use File > Open) to load it. What does it do? Hint, you’ll need to wait five seconds to find out.



RELAX



Just relax. At this point we don't expect you to read JavaScript like you grew up with it. In fact, all we want you to do right now is get a feel for what JavaScript looks like.

That said, you're not totally off the hook because we need to get your brain revved up and working. Remember that code on the previous page? Let's just walk through it to get a feel for what it might do:

```
setTimeout(wakeUpUser, 5000);  
function wakeUpUser() {  
    alert("Are you going to stare at this boring page forever?");  
}
```

A way to create reusable code and call it "wakeUpUser"?

Perhaps a way to count five seconds of time? Hint: 1000 milliseconds = 1 second.

Clearly a way to alert the user with a message.

THERE ARE NO DUMB QUESTIONS

Q: I've heard JavaScript is a bit of a wimpy language. Is it?

A: JavaScript certainly wasn't a power lifter in its early days, but its importance to the web has grown since then, and as a result, many resources (including brain power from some of the best minds in the business) have gone into supercharging the performance of JavaScript. But, you know what? Even before JavaScript was super fast, it was always a brilliant language. As you'll see, we're going to do some very powerful things with it.

Q: Is JavaScript related to Java?

A: Only by name. JavaScript was created during a time when Java was a red hot popular language, and the inventors of JavaScript capitalized on that popularity by making use of the Java name. Both languages borrow some syntax from programming languages like C, but other than that, they are quite different.

Q: Is JavaScript the best way to create dynamic web pages? Are there other technologies?

A: Not really. JavaScript is **the** language for programming in the browser. There are variants of JavaScript, like TypeScript, but those are not in common use, and even then the TypeScript code you write will be translated into JavaScript before it runs in the browser. With today's super

fast JavaScript environments and sophisticated APIs, JavaScript is here to stay and is the standard for programming in the browser.

Q: My friend is using JavaScript inside a music application, or at least he says he is. Is that possible?

A: Yes, JavaScript is breaking out of the browser as a general scripting language for many applications from graphics utilities to music applications and even to server-side programming. Your investment in learning JavaScript is likely to pay off in ways beyond web pages in the future.

Q: You say that many other languages are compiled. What exactly does that mean and why isn't JavaScript?

A: With conventional programming languages like C, C++ or Java, you compile the code before you execute it. Compiling takes your code and produces a machine efficient representation of it, usually optimized for runtime performance. Scripting languages are typically interpreted, which means that the browser runs each line of JavaScript code as it gets to it. Scripting languages place less importance on runtime performance, and are more geared towards tasks like prototyping, interactive coding and flexibility. This was the case with early JavaScript, and was why, for many years, the performance of JavaScript was not so great. There is a middle ground however; an interpreted language can be compiled on the fly, and that's the path browser manufacturers have taken with modern JavaScript.

In fact, with JavaScript you now have the conveniences of a scripting language, while enjoying the performance of a compiled language. By the way, we'll use the words *interpret*, *evaluate* and *execute* in this book. They have slightly different meanings in various contexts, but for our purposes, they all basically mean the same thing.

JavaScript, you've come a long way...

JavaScript 1.0

Netscape might have been before your time, but it was the first *real* browser company. Back in the mid-1990s browser competition was fierce, particularly with Microsoft, and so adding new, exciting features to the browser was a priority.

And towards that goal, Netscape wanted to create a scripting language that would allow anyone to add scripts to their pages. Enter LiveScript, a language developed in short order to meet that need. Now if you've never heard of LiveScript, that's because this was all about the time that Sun Microsystems introduced Java, and, as a result, drove their own stock to stratospheric levels. So, why not capitalize on that success and rename LiveScript to JavaScript? After all, who cares if they don't actually have anything to do with each other? Right?

Did we mention Microsoft? They created their own scripting language soon after Netscape did, named, um, JScript, and it was, um, quite similar to JavaScript. And so began the browser wars and a frustrating time for developers.

JavaScript 2015

After all that confusion, JavaScript finally grew up. ECMAScript, an official language definition for Javascript was born, and now serves as the standard language definition for all JavaScript implementations (in and out of the browser).

By 2015, JavaScript finally came of age and gained the respect of professional developers. Having a solid standard helped, along with serious commitment from browser-makers like Google, that pushed JavaScript into the professional limelight with Google Maps and other complex browser-based applications.

With all the new attention, many of the best programming language minds focused on improving JavaScript's interpreters and made improvements to its runtime performance. After JavaScript 2015, a major language update, we switched from official version numbers for language releases, to yearly updates.

JavaScript 2024

Today, JavaScript is *the* language of the Web. Pre-compilers make interpreting JavaScript code in web-pages blindingly fast. Syntax checkers, syntax-aware code editors, and robust browser-based debugging tools have professionalized web development. JavaScript is one of the most popular languages in the world, and has been implemented in environments as diverse as embedded systems, web servers, and music-making applications.

The language is fairly mature at this point. Most language updates are small, and incorporated quickly into browsers. Numerous JavaScript libraries and frameworks mean that you can do almost anything with JavaScript that you can do in any other language.

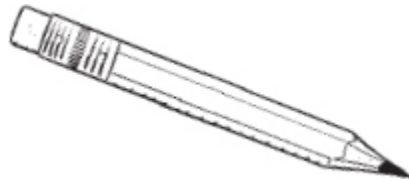
After a strange and contentious history, JavaScript has made it!

1995

2015

2024

SHARPEN YOUR PENCIL: LOOK HOW EASY IT IS TO WRITE JAVASCRIPT



You don't know JavaScript yet, but we bet you can make some good guesses about how JavaScript code works. Take a look at each line of code below and see if you can guess what it does. Write in your answers below. We've done one for you to get you started. If you get stuck, the answers are on the next page.

```
let price = 28.99;
let discount = 10;
let total =
    price - (price * (discount / 100));
if (total > 25) {
    freeShipping();
}
```

```
let count = 10;
while (count > 0) {
    juggle();
    count = count - 1;
}
```

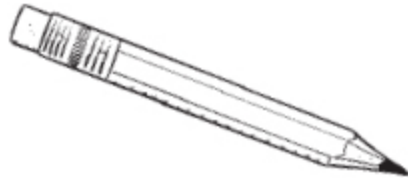


```
const dog = {name: "Rover", weight: 35};  
if (dog.weight > 30) {  
    alert("WOOF WOOF");  
} else {  
    alert("woof woof");  
}
```

```
let circleRadius = 20;  
let circleArea =  
    Math.PI * (circleRadius * circleRadius);
```

Create a variable named price, and assign the value 28.99 to it.

SHARPEN YOUR PENCIL SOLUTION: LOOK HOW EASY IT IS TO WRITE JAVASCRIPT



You don't know JavaScript yet, but we bet you can make some good guesses about how JavaScript code works. Take a look at each line of code below and see if you can guess what it does. Write in your answers below. We've done one for you to get you started. Here are our answers.

```
let price = 28.99;
let discount = 10;
let total =
    price - (price * (discount / 100));
if (total > 25) {
    freeShipping();
}
```

```
let count = 10;
while (count > 0) {
    juggle();
    count = count - 1;
}
```

```
const dog = {name: "Rover", weight: 35};  
if (dog.weight > 30) {  
    alert("WOOF WOOF");  
} else {  
    alert("woof woof");  
}
```

```
let circleRadius = 20;  
let circleArea =  
    Math.PI * (circleRadius * circleRadius);
```

Create a variable named price, and assign the value 28.99 to it
Create a variable named discount, and assign the value 10 to it
Compute a new price by applying a discount and then assign it to the variable total.
Compare the value in the variable total to 25. If it's greater...
...then do something with freeShipping.
End the if statement
Create a variable named count, and assign the value 10 to it.
As long as the variable count is greater than 0...
...do some juggling, and...
...reduce the value of count by 1 each time.
End the while loop
Create a constant dog with a name and weight.
If the dog's weight is greater than 30...
...alert "WOOF WOOF" to the browser's web page
Otherwise...
...alert "woof woof" to the browser's web page
End the if/else statement
Create a variable, circleRadius, and assign the value 20 to it.
Create a variable named circleArea...
...and assign the result of this expression to it (1256.6370614359173)



With HTML and CSS you can create some great looking pages. But once you know JavaScript, you can really expand on the kinds of pages you can create.

NOTE

Knowing JavaScript might increase the size of your paycheck too!

So much so, in fact, you might actually start thinking of your pages as applications (or even experiences!) rather than mere pages.

Now, you might be saying, “I already know that, why do you think I’m reading this book?” Well, we actually wanted to use this opportunity to have a little chat about learning JavaScript. If you already have a programming language or scripting language under your belt, then you have some idea of what lies ahead. However, if you’ve mostly been using HTML & CSS to date, you should know that there is something fundamentally different about learning a programming language.

With HTML & CSS what you’re doing is largely declarative—for instance, you’re declaring, say, that some text is a paragraph or that all the HTML elements in the “sale” class should be colored red. With JavaScript you’re adding *behavior* to the page, and to do that you need to describe computation. You’ll need to be able to describe things like, “compute the user’s score by summing up all the correct answers” or “do this action ten times” or “when the user clicks on that button play the you-have-won sound” or even “go off and get my latest tweet, and put it in this page.”

To do those things you need a language that is quite different from HTML or CSS. Let’s see how...

How to make a statement

When you create HTML you usually **mark up** text to give it structure; to do that you add elements, attributes and values to the text:


```
<h1 class="drink">Mocha Caffè Latte</h1>  
<p>Espresso, steamed milk and chocolate syrup,  
just the way you like it.</p>
```

With HTML we mark up text to create structure. Like, "I need a large heading called Mocha Caffè Latte; it's a heading for a drink. And I need a paragraph after that."

CSS is a bit different. With CSS you're writing a set of **rules**, where each rule selects elements in the page, and then specifies a set of styles for those elements:

```
h1.drink {
```

```
    color: brown;
```

```
}
```

```
p {
```

```
    font-family: sans-serif;
```

```
}
```

With CSS we write rules that use selectors, like h1.drink and p, to determine what parts of the HTML the style is applied to.

Let's make sure all drink headings are colored brown...

...and we want all the paragraphs to have a sans-serif type font.

With JavaScript you write **statements**. Each statement specifies a small part of a computation, and together, all the statements create the behavior of the page:

A set of statements. Each statement does a little bit of work, like declaring some variables to contain values for us.

```
let age = 25;
let name = "Owen";

if (age > 14) {
    alert("Sorry this page is for kids only!");
} else {
    alert("Welcome " + name + "!");
}
```

Here we create a variable to contain an age of 25, and we also need a variable to contain—or as we usually say, store—the value "Owen".

Or making decisions, such as: Is the age of the user greater than 14?

And if so alerting the user they are too old for this page.

Otherwise, we welcome the user by name, like this: "Welcome Owen!" (but since Owen is 25, we don't do that in this case.)

Variables and values

You might have noticed that JavaScript statements usually involve variables. Variables are used to store values. What kinds of values? Here are a few examples:

This statement declares a variable named winners and assigns a numeric value of 2 to it.

```
let winners = 2;
```



This one assigns a string of characters to the variable name (we call those "strings," for short).

```
let name = "Duke";
```



And this statement assigns the value false to the variable isEligible. We call true/false values "booleans."

```
let isEligible = false;
```



Pronounced "boo-lee-ans."

There are other values that variables can hold beyond numbers, strings and booleans, and we'll get to those soon enough, but, no matter what a variable contains, we create all variables the same way. Let's take a little closer look at how to declare a variable:

The diagram shows the code `let winners = 2;` with several handwritten annotations and arrows explaining its parts:

- An arrow points to `let` with the text: "We start with the let keyword when declaring a variable whose value can change."
- An arrow points to the space between `let` and `winners` with the text: "Even if JavaScript doesn't complain when you leave off the let, you should declare your variables. We'll tell you why later..."
- An arrow points to `winners` with the text: "Next we give the variable a name."
- An arrow points to `=` with the text: "And, optionally, we assign a value to the variable by adding an equals sign followed by the value."
- An arrow points to `2` with the same text: "And, optionally, we assign a value to the variable by adding an equals sign followed by the value."
- An arrow points to `;` with the text: "We always end an assignment statement with a semicolon."

We say optionally, because if you want, you can create a variable without an initial value, and then assign it a value later. To create a variable without an initial value, just leave off the assignment part, like this:

The diagram shows the code `let losers;` with a handwritten annotation and arrow:

- An arrow points to the code with the text: "By leaving off the equals sign and value you're just declaring the variable for later use."

No value?! What am
I supposed to do now?!
I'm so humiliated.



losers

BRAIN POWER



If you copy the `start`, `stop`, and `drive` methods into the `chevy` and `cadi` objects we created earlier, what do you have to change to make the methods work correctly?

If you declare a variable without a value, such as:

We're using `let` to declare a new variable.

We're calling the variable `flavor`.

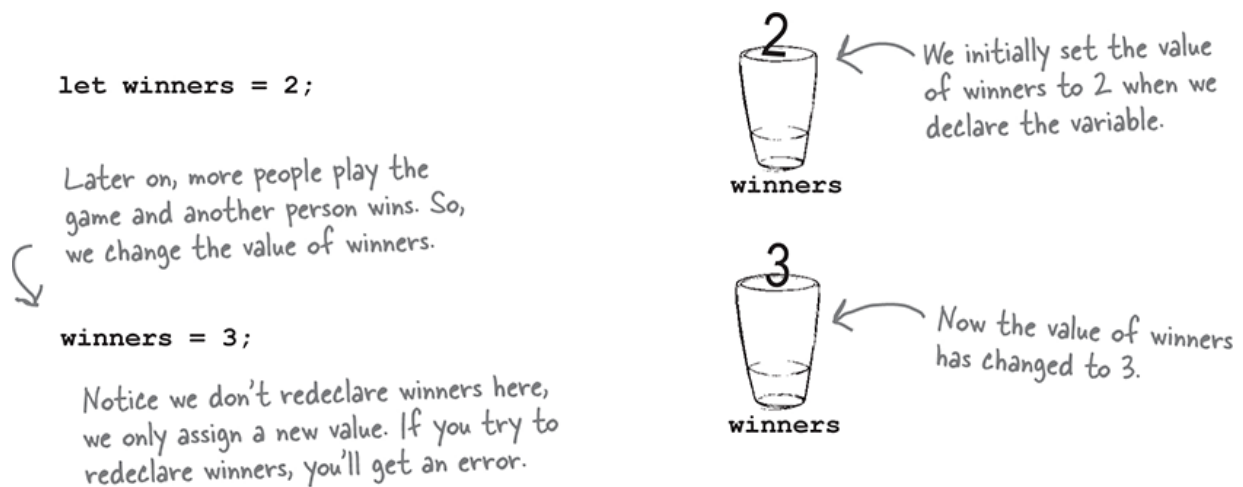
`let flavor;`

But we're not giving it an initial value.

What value do you think JavaScript is assigning to this variable?

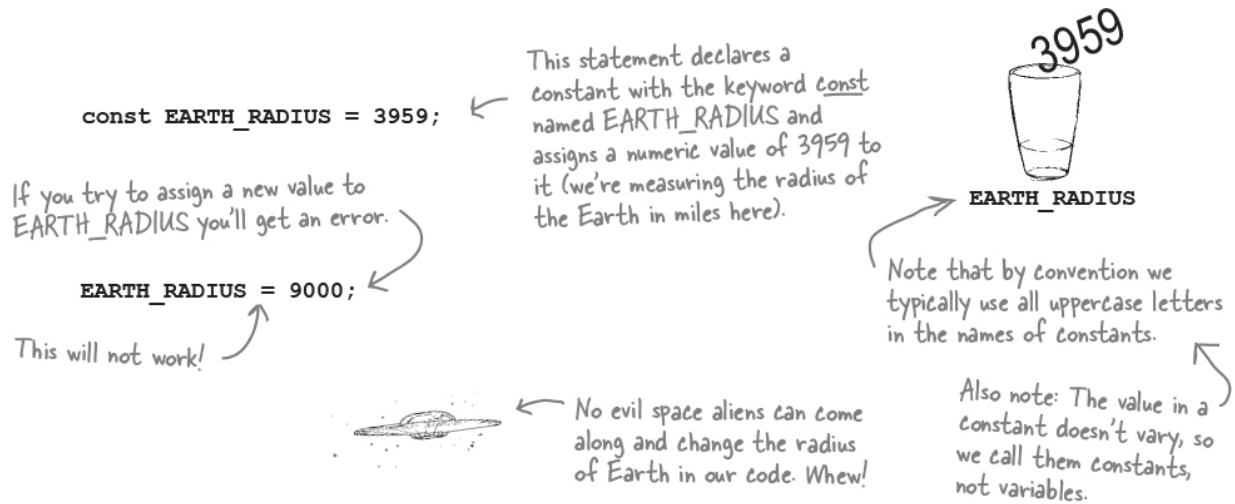
Constants, another kind of variable

So far, we've used the keyword **let** to declare our variables. And that's typically what we want to do with variables whose values can *vary*, or in other words, change their value over time. For instance, if we use **let** to declare the variable **winners**, assign it the value 2, we can change the value in **winners** later to be 3 if another winner comes along:

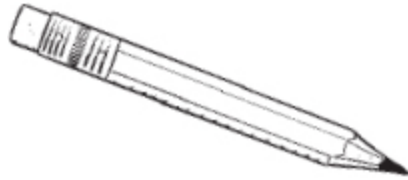


Sometimes, however, we do not want the values in our variables to vary at all. There are situations in which we might want to give a name to a value that we'll use in our code, but we don't ever want that value to change. Here's a good example: the radius of planet Earth. It might be handy to assign this value to a variable so we can use **EARTH_RADIUS** instead of the number in our code. We don't want anyone to come along and change this value accidentally, so how can we make sure the value of

EARTH_RADIUS never changes? We can use a *constant* instead of a variable, like this:



SHARPEN YOUR PENCIL



Identify that declarations below that you think are best suited for `let` and for `const`. Remember, while all uppercase text is a convention for constants, it isn't always adhered to. We've done a couple for you:

<u>const</u>	<code>DISTANCE_TO_MOON = 238900;</code>
<u>let</u>	<code>last_guess = 0;</code>
<u> </u>	<code>SCREEN_WIDTH = 1024;</code>
<u> </u>	<code>counter = 100;</code>
<u> </u>	<code>firstUSPresident = "Washington";</code>
<u> </u>	<code>fluxCapacitorReading = "System Normal";</code>

Back away from that keyboard!

You know variables have a name, and you know they have a value.

You also know some of the things a variable can hold are numbers, strings and boolean values.

But what can you call your variables? Is any name okay? Well no, but the rules around creating variable names are simple: just follow the two rules below to create valid variable names:

- **❶ Start your variables with a letter, an underscore or a dollar sign.**
- **❷ After that, use as many letters, numeric digits, underscores or dollar signs as you like.**

Oh, and one more thing; we really don't want to confuse JavaScript by using any of the built-in *keywords*, like **let** or **function** or **false**, so consider those off limits for your own variable names. We'll get to some of these keywords and what they mean throughout the rest of the book, but here's a list to take a quick look at:

- | | |
|------------|-----------|
| • break | • default |
| • case | • delete |
| • catch | • do |
| • class | • else |
| • const | • enum |
| • continue | • export |
| • debugger | • extends |

- false
- finally
- for
- function
- if
- implements
- import
- in
- instanceof
- interface
- let
- new
- package
- private
- protected
- public
- return
- static
- super
- switch
- this
- throw
- true
- try
- typeof
- var
- void
- while
- with
- yield



THERE ARE NO DUMB QUESTIONS

Q: What's a keyword?

A: A keyword is a reserved word in JavaScript. JavaScript uses these reserved words for its own purposes, and it would be confusing to you and the browser if you started using them for your variables and constants.

Q: Why aren't we using var to declare our variables?

A: The var keyword is no longer recommended and has been largely replaced by let and const. These keywords work slightly differently to var and have some benefits over var that we'll come back to later.

Q: What if I used a keyword as part of my variable name? For instance, can I have a variable named ifOnly (that is, a variable that contains the keyword if)?

A: You sure can, just don't match the keyword exactly. It's also good to write clear code, so in general you wouldn't want to use something like `elze`, which might be confused with `else`.

Q: Is JavaScript case sensitive? In other words, are myvariable and MyVariable the same thing?

A: If you're used to HTML markup you might be used to case insensitive languages; after all, `<head>` and `<HEAD>` are treated the same by the

browser. With JavaScript however, case matters for variables, constants, keywords, function names and pretty much everything else, too. So pay attention to your use of upper- and lowercase.

WEBVILLE TIMES

How to avoid those embarrassing naming mistakes

You've got a lot of flexibility in choosing your variable and constant names, so here are a few Webville tips to make your naming easier:

Choose names that mean something.

Variable names like `_m`, `$`, `r` and `foo` might mean something to you but they are generally frowned upon in Webville. Not only are you likely to forget them over time, your code will be much more readable with names like `angle`, `currentPressure` and `passedExam`.

Use all uppercase name for constants.

This is a common convention in programming.

Use "camel case" when creating multiword variable names.

At some point you're going to have to decide how you name a variable that represents, say, a two-headed dragon with fire. How? Just use camel case, in which you capitalize the first letter of each word (other than the first): `twoHeadedDragonWithFire`. Camel case is easy to form, widely spoken in Webville and gives you enough flexibility to create as specific a variable name as you need. There are other schemes too, but this is one of the more commonly used (even beyond JavaScript). For constants, use an underscore between words.

Use variables that begin with `_` and `$` only with very good reason.

Variables that begin with `$` are usually reserved for JavaScript libraries and while some authors use variables beginning with `_` for various conventions, we recommend you stay away from both unless you have very good reason (you'll know if you do).

Be safe.

Be safe in your variable naming; we'll cover a few more tips for staying safe later in the book, but for now be clear in your naming, avoid keywords, and always use `let` when declaring a variable and `const` when declaring a constant.

SYNTAX FUN



- Each statement ends in a semicolon.

```
x = x + 1;
```

- A single line comment begins with two forward slashes. Comments are just notes to you or other developers about the code. They aren't executed.

```
// I'm a comment
```

- Whitespace doesn't matter (almost everywhere).

```
x      =      2233;
```

- Surround strings of characters with double quotes (or single, both work, just be consistent).

```
"You rule!"  
'And so do you!'
```

- Don't use quotes around the boolean values true and false.

```
rockin = true;
```

- Variables don't have to be given a value when they are declared: `let width;`
 - JavaScript, unlike HTML markup, is case sensitive, meaning upper- and lowercase matters. The variable `counter` is different from the variable `Counter`.
-

BE THE BROWSER

Below, you'll find JavaScript code with some mistakes in it. Your job is to play like you're the browser and find the errors in the code. After you've done the exercise look at the end of the chapter to see if you found them all.



A

```
// Test for jokes
const joke = "JavaScript walked into a bar....";
let toldJoke = "false";
let $punchline =
    "Better watch out for those semi-colons."
let %entage = 20;
let result

if (toldJoke == true) {
    Alert($punchline);
} else
    alert(joke);
}
joke = "Knock, knock. Who's there? JavaScript....";
```

Don't worry too much about what this JavaScript does for now; just focus on looking for errors in variables and syntax.



B

```
\\ Movie Night
let zip code = 98104;
const joe'sFavoriteMovie = Forbidden Planet;
let movieTicket$ = 9;

if (movieTicket$ >= 9) {
    alert("Too much!");
} else {
    alert("We're going to see " + joe'sFavoriteMovie);
}
```

Express yourself

To truly express yourself in JavaScript you need **expressions**. Expressions evaluate to values. You've already seen a few in passing in our code examples. Take the expression in this statement for instance:

Here's a JavaScript statement that assigns the result of evaluating an expression to the variable `total`.

This expression evaluates to a price reduced by a discount that is a percent of the price. So if your price is 10 and the discount is 20, we get 8 as a result.

We use `*` for multiply and `/` for divide.

```
let total = price - (price * (discount / 100));
```

Here's our variable `total`.

And the assignment.

And this whole thing is an expression.

If you've ever taken a math class, balanced your checkbook or done your taxes, we're sure these kinds of numeric expressions are nothing new.

There are also string expressions; here are a few:

```
"Dear " + "Reader" + ", "
```

This adds together, or concatenates, these strings to form a new string "Dear Reader,".

```
"super" + "cali" + youKnowTheRest
```

Same here, except we have a variable that contains a string as part of the expression. This evaluates to "supercalifragilisticexpialidocious".

```
phoneNumber.substring(0,3)
```

Just another example of an expression that results in a string. We'll get to exactly how this works later, but this returns the area code of a US phone number string.

We also have expressions that evaluate to **true** or **false**, otherwise known as boolean expressions. Work through each of these to see how you get true or false from them:

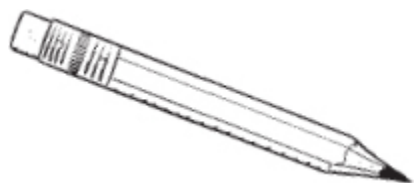
`age < 14` ← If a person's age is less than 14 this is true, otherwise it is false. We could use this to test if someone is a child or not.

`cost >= 3.99` ← If the cost is 3.99 or greater, this is true. Otherwise it's false. Get ready to buy on sale when it's false!

`animal == "bear"` ← This is true when animal contains the string "bear". If it does, beware!

And expressions can evaluate to a few other types; we'll get to these later in the book. For now, the important thing is to realize all these expressions evaluate to something: a value that is a number, a string or a boolean. Let's keep moving and see what that gets you!

SHARPEN YOUR PENCIL



Get out your pencil and put some expressions through their paces. For each expression below, compute its value and write in your answer. Yes, **WRITE IN...** forget what your Mom told you about writing in books and scribble your answer right in this book! Be sure to check your answers at the end of the chapter.

Can you say "Celsius to Fahrenheit calculator"?

`(9 / 5) * temp + 32`

What is the result when temp is 10? _____

`color == "orange"`

This is a boolean expression. The `==` operator tests if two values are equal to each other.

Is this expression true or false when color has the value "pink"? _____
Or has the value "orange"? _____

`name + ", " + "you've won!"`

What value does this compute to when name is "Martha"? _____

`yourLevel > 5`

This tests if the first value is greater than the second. You can also use `>=` to test if the first value is greater than or equal to the second.

When yourLevel is 2, what does this evaluate to? _____

When yourLevel is 5, what does this evaluate to? _____

When yourLevel is 7, what does this evaluate to? _____

`(level * points) + bonus`

Okay, level is 5, points is 30,000 and bonus is 3300. What does this evaluate to? _____

`color != "orange"`

Is this expression true or false when color has the value "pink"? _____

The `!=` operator tests if two values are NOT equal to each other.

Extra CREDIT!

`1000 + "108"`

Are there a few possible answers?
Only one is correct. Which would you choose? _____

SERIOUS CODING



Did you notice that the `=` operator is used in assignments, while the `==` operator tests for equality? That is, we use one equal sign to assign values to variables. We use two equal signs to test if two values are equal to each other. Substituting one for the other is a common coding mistake.

```
while (juggling) {  
    keepBallsInAir();  
}
```



Doing things more than once

You do a lot of things more than once:

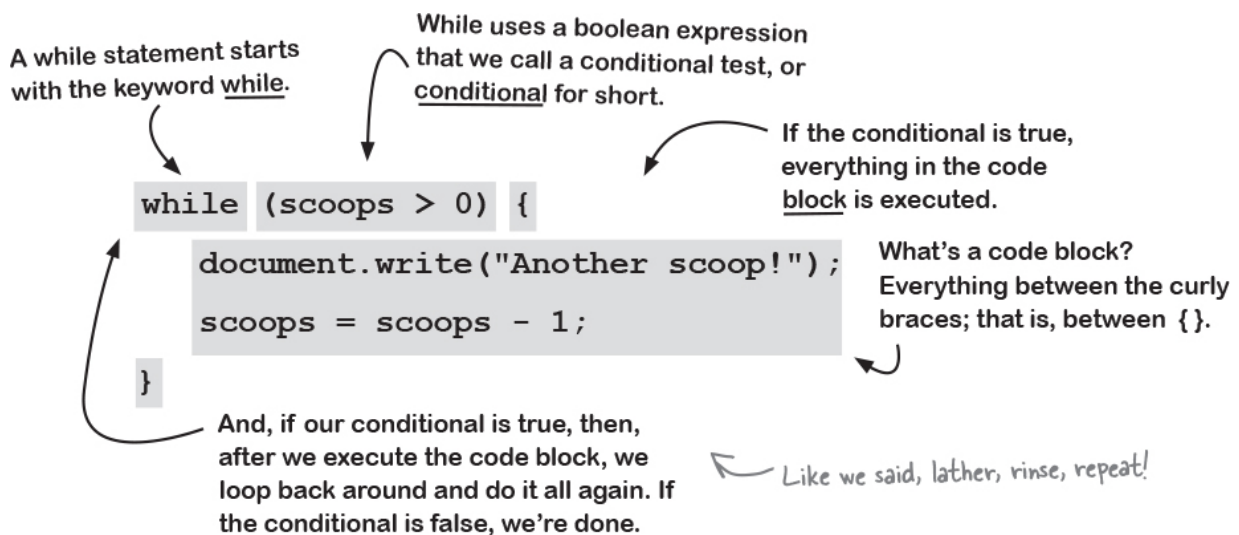
Lather, rinse, repeat...

Wax on, wax off...

Eat candies from the bowl until they're all gone.

Of course you'll often need to do things in code more than once, and JavaScript gives you a few ways to repeatedly execute code in a loop: **while**, **for**, **for in** and **forEach**. Eventually, we'll look at all these ways of looping, but let's focus on **while** for now.

We just talked about expressions that evaluate to boolean values, like `scoops > 0`, and these kinds of expressions are the key to the while statement. Here's how:



How the while loop works

Seeing as this is your first while loop, let's trace through a round of its execution to see exactly how it works*. Notice we've added a declaration for scoops to declare it, and initialize it to the value 5.

Now let's start executing this code. First we set scoops to five.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

After that we hit the while statement. When we evaluate a while statement the first thing we do is evaluate the conditional to see if it's true or false.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

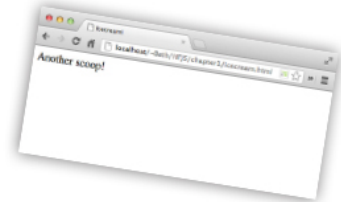
Is scoops greater
than zero? Looks
like it to us!



Because the conditional is true, we start executing the block of code.

The first statement in the body writes the string “Another scoop!
” to the browser.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



* To follow along, grab the code for this chapter from <http://wickedlysmart.com/hfjs> and drag the file icecream.html into your browser.

The next statement subtracts one from the number of scoops and then sets scoops to that new value, four.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



That’s the last statement in the block, so we loop back up to the conditional and start over again.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

Evaluating our conditional again, this time scoops is four. But that's still more than zero.

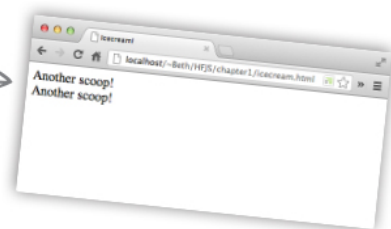
```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

Still plenty left!



Once again we write the string “Another scoop!
” to the browser.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



The next statement subtracts one from the number of scoops and sets scoops to that new value, which is three.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

2 scoops gone,
3 left!



That's the last statement in the block, so we loop back up to the conditional and start over again.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

Evaluating our conditional again, this time scoops is three. But that's still more than zero.

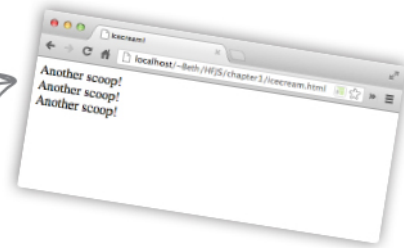
```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

Still plenty left!



Once again we write the string “Another scoop!
” to the browser.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



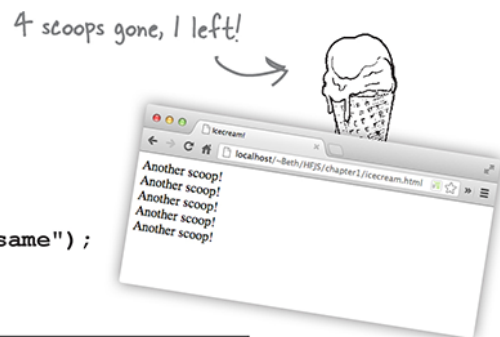
And as you can see, this continues... each time we loop, we decrement (reduce scoops by 1), write another string to the browser, and keep going.

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



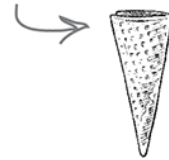
And continues...

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



Until the last time... this time something's different. Scoops is zero, and so our conditional returns false. That's it folks; we're not going to go through the loop anymore, we're not going to execute the block. This time, we bypass the block and execute the statement that follows it.

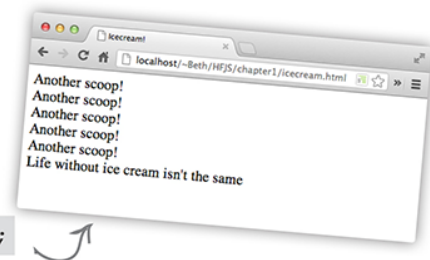
5 scoops gone, 0 left!



```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```

Now we execute the other document.write, and write the string “Life without ice cream isn’t the same”. We’re done!

```
let scoops = 5;
while (scoops > 0) {
  document.write("Another scoop!<br>");
  scoops = scoops - 1;
}
document.write("Life without ice cream isn't the same");
```



```
if (cashInWallet > 5) {
  order = "I'll take the works:
cheeseburger, fries and a coke";
} else {
```

```
    order = "I'll just have a glass of  
water";  
}
```

Making decisions with JavaScript

You've just seen how you use a conditional to decide whether to continue looping in a `while` statement. You can also use boolean expressions to make decisions in JavaScript with the `if` statement. The `if` statement executes its code block only if a conditional test is true. Here's an example:

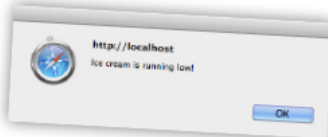
Here's the `if` keyword, followed by a conditional and a block of code.

```
if (scoops < 3) {  
    alert("Ice cream is running low!");  
}
```

This conditional tests to see if we're down to fewer than three scoops.

And if we've got fewer than three left, then we execute the `if` statement's code block.

`alert` takes a string and displays it in a popup dialog in your browser. Give it a try!



With an `if` statement we can also string together multiple tests by adding on one or more `else if`'s, like this:

else if's, like this:

```
if (scoops >= 5) {  
    alert("Eat faster, the ice cream is going to melt!");  
} else if (scoops < 3) {  
    alert("Ice cream is running low!");  
}
```

We can have one test, and then another test with if/else if

Add as many tests with "else if" as you need, each with its own associated code block that will be executed when the condition is true.

And, when you need to make LOTS of decisions

You can string together as many `if / else` statements as you need, and if you want one, even a final catch-all `else`, so that if all conditions fail, you can handle it. Like this:

```
if (scoops >= 5) {  
    alert("Eat faster, the ice cream is going to melt!");  
} else if (scoops == 3) {  
    alert("Ice cream is running low!");  
} else if (scoops == 2) {  
    alert("Going once!");  
} else if (scoops == 1) {  
    alert("Going twice!");  
} else if (scoops == 0) {  
    alert("Gone!");  
} else {  
    alert("Still lots of ice cream left, come and get it.");  
}
```

In this code we check to see if there are five or more scoops left...

...or if there are precisely three left...

...or if there are 2, 1 or 0, and then we provide the appropriate alert.

And if none of the conditions above are true, then this code is executed.



THERE ARE NO DUMB QUESTIONS

Q: What exactly is a code block?

A: Syntactically, a code block (which we usually just call a block) is a set of statements, which could be one statement, or as many as you like, grouped together between curly braces. Once you've got a block of code, all the statements in that block are treated as a group to be executed together in sequence. For instance, all the statements within the block in a while statement are executed if the condition of the while is true. The same holds for a block in an if or else if.

Q: I've seen code where the conditional is a variable with a value that isn't a boolean, like a string or a number. How does that work?

A: We'll be covering that a little later, but the short answer is JavaScript is quite flexible in what it thinks is a true or false value. For instance, any variable that holds a (non-empty) string is considered true, but a variable that hasn't been set to a value is considered false. We'll get into these details soon enough.

Q: You've said that expressions can result in things other than numbers, strings and booleans. Like what?

A: Right now we're concentrating on what are known as the *primitive types*, that is, numbers, strings and booleans. Later we'll take a look at more

complex types, like arrays, which are collections of values, objects and functions.

Q: Where does the name boolean come from?

A: Booleans are named after George Boole, an English mathematician who invented Boolean logic. You'll often see boolean written "Boolean," to signify that these types of variables are named after George.

CODE MAGNETS



A JavaScript program is all scrambled up on the fridge. Can you put the magnets back in the right places to make a working JavaScript program to produce the output shown below? Check your answer at the end of the chapter before you go on.

Arrange these magnets to make a working JavaScript program.

```
document.write("Happy Birthday dear " + name + ",<br>");
```

```
document.write("Happy Birthday to you.<br>");
```

```
let i = 0;
```

```
const name = "Joe";
```

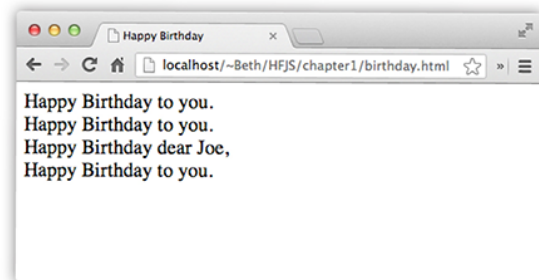
```
i = i + 1;
```

```
}
```

```
document.write("Happy Birthday to you.<br>");
```

```
while (i < 2) {
```

Your unscrambled program should produce this output.



↑
Use this space for your re-arranged magnets.

Reach out and communicate with your user

We've been talking about making your pages more interactive, and to do that you need to be able to communicate with your user. As it turns out there are a few ways to do that, and you've already seen some of them.

Let's get a quick overview and then we'll dive into these in more detail throughout the book:

Create an alert

As you've seen, the browser gives you a quick way to alert your users through the `alert` function. Just call `alert` with a string containing your alert message, and the browser will give your user the message in a nice dialog box. A small confession though: we've been overusing this because it's easy; `alert` really should be used only when you truly want to stop everything and let the user know something.

Write directly into your document

Think of your web page as a document (that's what the browser calls it). You can use a function `document.write` to write arbitrary HTML and content into your page at any point. In general, this is considered bad form, although you'll see it used here and there. We've used it a bit in this chapter too because it's an easy way to get started.

NOTE

We're using these three methods in this chapter.

Use the console

Every JavaScript environment also has a console that can log messages from your code. To write a message to the console's log you use the function `console.log` and hand it a string that you'd like printed to the log (more details on using console log in a second). You can view `console.log` as a great tool for troubleshooting your code, but typically your users will never see your console log, so it's not a very effective way to communicate with them.

NOTE

The console is a really handy way to help find errors in your code! If you've made a typing mistake, like missing a quote, JavaScript will usually give you an error in the console to help you track it down.

Directly manipulate your document

This is the big leagues; this is the way you want to be interacting with your page and users—using JavaScript you can access your actual web page, read & change its content, and even alter its structure and style! This all happens by making use of your browser's *document object model* (more on that later). As you'll see, this is the best way to communicate with your user. But, using the document object model requires knowledge of how your page is structured and of the programming interface that is used to

read and write to the page. We'll be getting there soon enough. But first, we've got some more JavaScript to learn.

NOTE

This is what we're working towards. When you get there you'll be able to read, alter and manipulate your page in any number of ways.

WHO DOES WHAT

All our methods of communication have come to the party with masks on. Can you help us unmask each one? Match the descriptions on the right to the names on the left. We've done one for you.

document.write

I'll stop your user in his tracks and deliver a short message. The user has to click on "ok" to go further.

console.log

I can insert a little HTML and text into a document. I'm not the most elegant way to get a message to your users, but I work on every browser.

alert

Using me you can totally control a web page: get values that a user typed in, alter the HTML or the style, or update the content of your page.

document object model

I'm just here for simple debugging purposes. Use me and I can write out information to a special developer's console.

A closer look at console.log

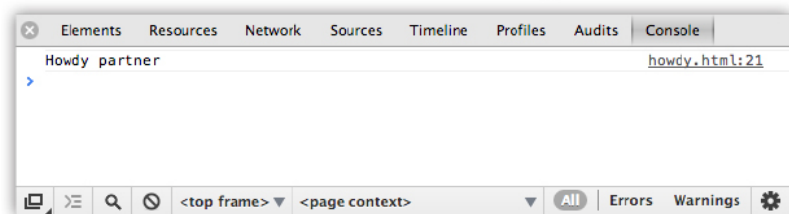
Let's take a closer look at how `console.log` works so we can use it in this chapter to see the output from our code, and throughout the book to inspect the output of our code and debug it. Remember though, the console is not a browser feature most casual users of the web will encounter, so you won't want to use it in the final version of your web page. Writing to the console log is typically done to troubleshoot as you develop your page. That said, it's a great way to see what your code is doing while you're learning the basics of JavaScript. Here's how it works:

```
let message = "Howdy" + " " + "partner";  
console.log(message);
```

Take any old string...

...and give it to console.log, and it will be shown in the browser's console, like this.

The console contains all the output logged by your code.



THERE ARE NO DUMB QUESTIONS

Q: I get that `console.log` can be used to output strings, but what exactly is it? I mean why are the “console” and the “log” seperated by a period?

A: Ah, good point. We’re jumping ahead a bit, but think of the console as an object that does things, console-like things. One of those things is logging, and to tell the console to log for us, we use the syntax “`console.log`” and pass it our output in between parentheses. Keep that in the back of your mind; we’re coming back to talk a lot more about objects a little later in the book. For now, you’ve got enough to use `console.log`.

Q: Can the console do anything other than just log?

A: Yes, but typically people just use it to log. There are a few more advanced ways to use log (and console), but they tend to be browser-specific. Note that console is something all modern browsers supply, but it isn’t part of any formal specification.

Q: Uh, console looks great, but where do I find it? I’m using it in my code and I don’t see any output!

A: In most browsers you have to explicitly open the console window. Check out the next page for details.

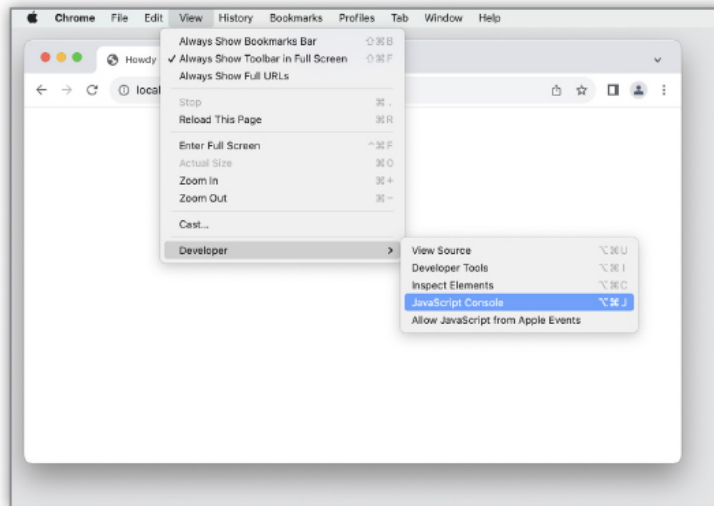
Opening the console

Every browser has a slightly different implementation of the console. And, to make things even more complicated, the way that browsers implement the console changes fairly frequently—not in a huge way, but enough so that by the time you read this, your browser’s console might look a bit different from what we’re showing here.

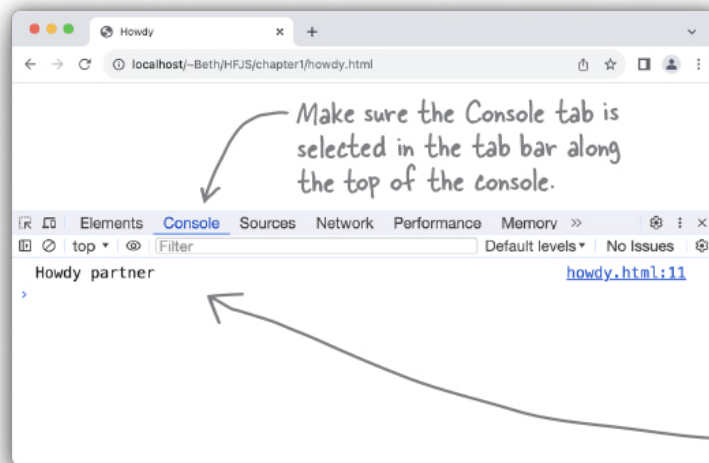
So, we’re going to show you how to access the console in the Chrome browser (version 120) on the Mac, and we’ll put instructions on how to access the console in all the major browsers online at <http://wickedlysmart.com/hfjsconsole>. Once you get the hang of the console in one browser, it’s fairly easy to figure out how to use it in other browsers too, and we encourage you to try using the console in at least two browsers so you’re familiar with them.

NOTE

Note: You don’t need to type the Howdy code in. We’re just learning where the console is. We’ll start typing in code in just a sec...



To access the console in Chrome (on the Mac), use the View > Developer > JavaScript Console menu.



The console will appear in the bottom part of your browser window.

Don't worry about what these other tabs are for. They're useful, but the most important one now is Console, so we can see console.log messages from our code.

You should see any messages you give to console.log in your code displayed in the window here.

Coding a Serious JavaScript Application

Let's put all these new JavaScript skills and `console.log` to good use with something practical. We need some variables, a `while` statement, some `if` statements with `else`s. Add a little more polish and we'll have a super-serious business application before you know it. But, before you

look at the code, think to yourself how you'd code that classic favorite, "99 bottles of rootbeer."



```
const word = "bottles";
let count = 99;
while (count > 0) {
  console.log(count + " " + word + " of rootbeer");
  console.log(count + " " + word + " of rootbeer");
  console.log("Take one down, pass it around,");
  count = count - 1;
  if (count > 0) {
    console.log(count + " " + word + " of rootbeer");
  } else {
    console.log("No more " + word + " of rootbeer");
  }
}
```

```
}  
}
```

BRAIN POWER



There’s still a little flaw in our code. It runs correctly, but the output isn’t 100% perfect. See if you can find the flaw, and fix it.



Isn't it about
time to put code in an
actual web page? Or are we just
going to keep writing answers on
the pages of this book?



Good point! Yes, it's time. Before we got there we wanted to make sure you had enough JavaScript under your belt to make it interesting. That said, you already saw in the beginning of this chapter that you add JavaScript to your HTML just like you add CSS; that is, you just add it inline with the appropriate `<script>` tags around it.

Now, like CSS, you can also place your JavaScript in files that are external to your HTML.

Let's first get this serious business application into a page, and then after we've thoroughly tested it, we'll move the JavaScript out to an external file.

TEST DRIVE



Okay, let's get some code in the browser... follow the instructions below and get your serious business app launched! You'll see our result below:

NOTE

To download all the code and sample files for this book, please visit

<http://wickedlysmart.com/hfjs>.

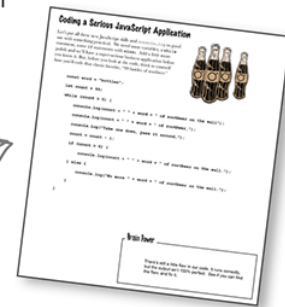
- 1 Check out the HTML below; that's where your JavaScript's going to go. Go ahead and type in the HTML and then place the JavaScript from two pages back in between the `<script>` tags. You can use an editor like Notepad (Windows) or TextEdit (Mac), making sure you are in plain text mode. Or, if you have a favorite HTML editor, like Dreamweaver, Coda or WebStorm, you can use that too.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>My First JavaScript</title>
  </head>
  <body>
    <script>

  </script>
</body>
</html>
```

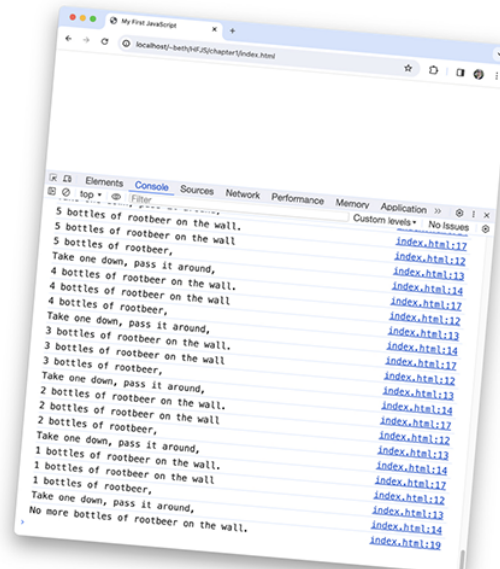
← Type this in.

Here are the `<script>` tags. At this point you know that's where you should put your code.



- 2 Save the file as "index.html".
- 3 Load the file into your browser. You can either drag the file right on top of your browser window, or use the File > Open (or File > Open File) menu option in your favorite browser.
- 4 You won't see anything in the web page itself because we're logging all the output to the console, using `console.log`. So open up the browser's console, and congratulate yourself on your serious business application.

Here's our test run of this code. The code creates the entire lyrics for the 99 bottles of rootbeer song and logs the text to the browser's console.



How do I add code to my page? (let me count the ways)

You already know you can add the `<script>` element with your JavaScript code to the `<head>` or `<body>` of your page, but there are a

couple of other ways to add your code to a page. Let's check out all the places you can put JavaScript (and why you might want to put it one place over another):

You can place your code inline, in the <head> element. The most common way to add code to your pages is to put a <script> element in the <head>. Sure, it makes your code easy to find and seems to be a logical place for your code, but it's not always the best place. Why? Read on...

Or, you can add your code inline in the body of the document. To do this, enclose your JavaScript code in the <script> element and place it in the <body> of your page (typically at the end of the body).

This is a little better. Why? When your browser loads a page, it loads everything in your page's <head> before it loads the <body>. So, if your code is in the <head>, users might have to wait a while to see the page. If the code is loaded after the HTML in the <body>, users will get to see the page content while they wait for the code to load.

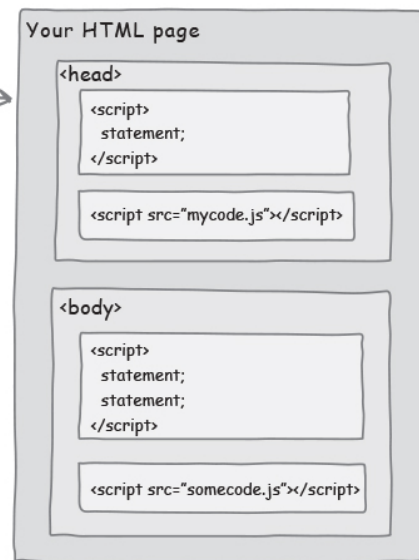
Still, is there a better way? Read on...

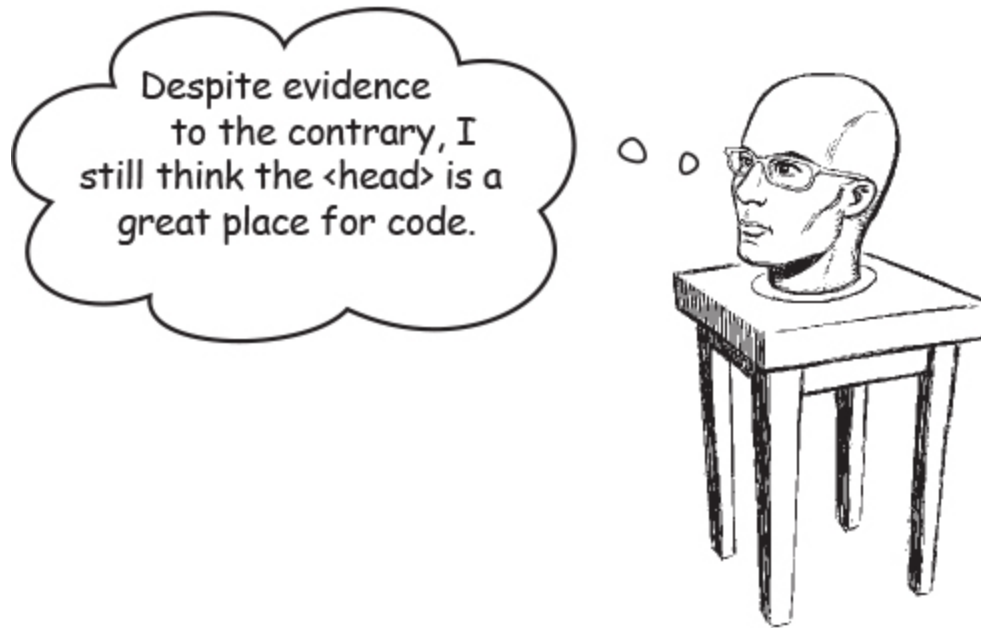
Or, put your code in its own file and link to it from the <head>.

This is just like linking to a CSS file. The only difference is that you use the src attribute of the <script> tag to specify the URL to your JavaScript file.

When your code is in an external file, it's easier to maintain (separately from the HTML) and can be used across multiple pages. But this method still has the drawback that all the code needs to be loaded before the body of the page. Is there a better way? Read on...

Finally, you can link to an external file in the body of your page. Ahhh, the best of both worlds. We have a nice, maintainable JavaScript file that can be included in any page, and it's referenced from the bottom of the body of the page, so it's only loaded after the body of the page. Not bad.





We're going to have to separate you two

Going separate ways hurts, but we know we have to do it. It's time to take your JavaScript and move it into its own file. Here's how you do that...

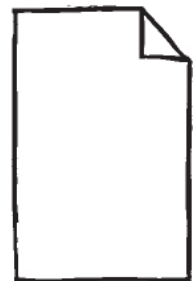
- ① Open index.html and select all the code; that is, everything between the `<script>` tags. Your selection should look like this:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>My First JavaScript</title>
  </head>
  <body>
    <script>
      const word = "bottles";
      let count = 99;
      while (count > 0) {
        console.log(count + " " + word + " of rootbeer on the wall");
        console.log(count + " " + word + " of rootbeer,");
        console.log("Take one down, pass it around,");
        count = count - 1;
        if (count > 0) {
          console.log(count + " " + word + " of rootbeer on the wall.");
        } else {
          console.log("No more " + word + " of rootbeer on the wall.");
        }
      }
    </script>
  </body>
</html>
```

Select just the code, not the `<script>` tags; you won't need those where you're going...

- ② Now create a new file named "code.js" in your editor, and place the code into it. Then save "code.js".

code.js



- ③ Now we need to place a reference to the "code.js" file in "index.html" so that it's retrieved and loaded when the page loads. To do that, delete the JavaScript code from "index.html", but leave the <script> tags. Then add a src attribute to your opening <script> tag to reference "code.js".

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>My First JavaScript</title>
  </head>
  <body>
    <script src="code.js">
    </script>
  </body>
</html>
```

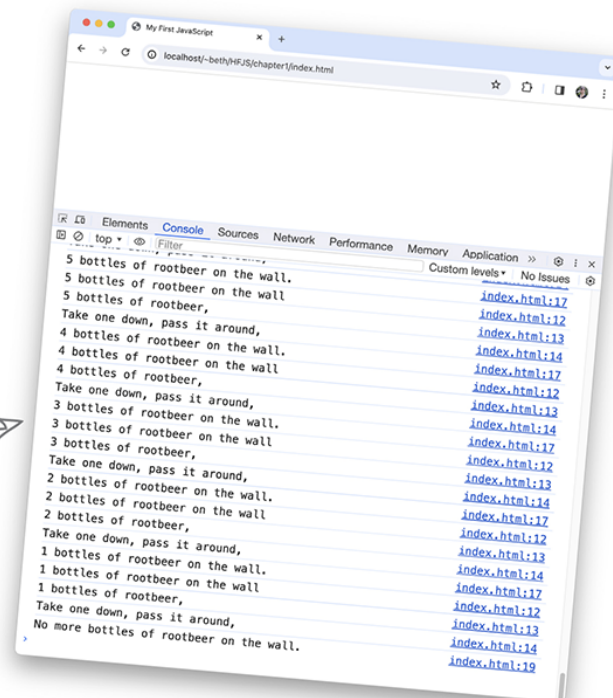
Use the src attribute of the <script> element to link to your JavaScript file.

Where your code was.

Believe it or not we still need the ending <script> tag, even if there is no code between the two tags.

- ④ That's it, the surgery is complete. Now you need to test it. Reload your "index.html" page and you should see exactly the same result as before. Note that by using a src="code.js", we're assuming that the code file is in the same directory as the HTML file.

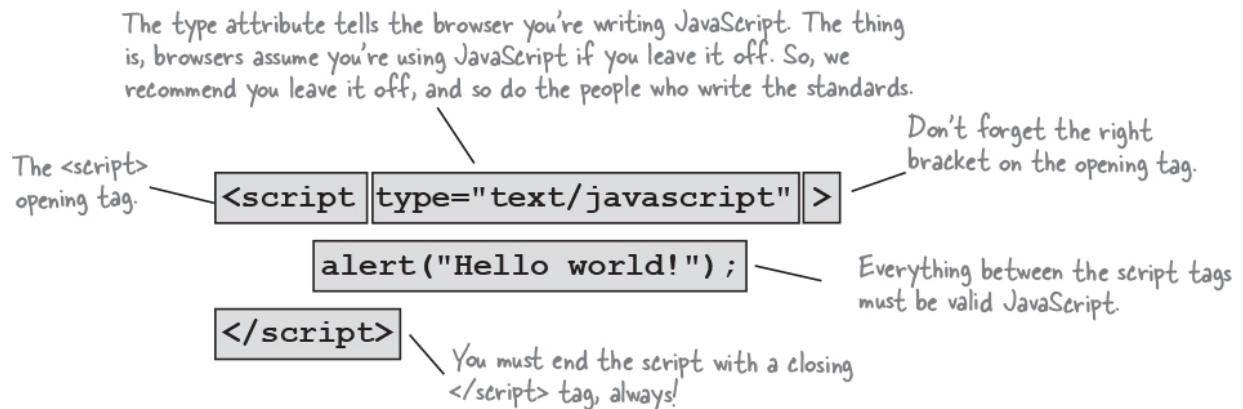
You should get the same result as before. But now your HTML and JavaScript are in separate files. Doesn't that just feel cleaner, more manageable, more stress-free already?



ANATOMY OF A SCRIPT ELEMENT



You know how to use the `<script>` element to add code to your page, but just to really nail down the topic, let's review the `<script>` element to make sure we have every detail covered:



And when you are referencing a separate JavaScript file from your HTML, you'll use the `<script>` element like this:

Add a src attribute to specify the URL of the JavaScript file.

```
<script src="myJavaScript.js">
```

```
</script>
```

Use ".js" as the extension on JavaScript files.

When referencing a separate JavaScript file, you don't put any JavaScript in the content of the <script> element.

Again, don't forget the closing </script> tag. You need it even when you're linking to an external file.

WATCH IT!



You can't use inline and external together.

If you try throwing some quick code in between those `<script>` tags when you're already using a `src` attribute, it won't work. You'll need two separate `<script>` elements.

```
<script src="goodies.js">  
  let x = "quick hack";  
</script>
```

WRONG

JAVASCRIPT EXPOSED: THIS WEEK'S INTERVIEW: GETTING TO KNOW JAVASCRIPT



Head First: Welcome JavaScript. We know you're super-busy out there, working on all those web pages, so we're glad you could take time out to talk to us.

JavaScript: No problem. And, I *am* busier than ever these days; people are using JavaScript on just about every page on the Web nowadays, for everything from simple menu effects to full blown games. It's nuts!

Head First: That's amazing given that just a few years ago, someone said that you were just a "half-baked, wimpy scripting language" and now you're everywhere.

JavaScript: Don't remind me. I've come a long way since then, and many great minds have been hard at work making me better.

Head First: Better how? Seems like your basic language features are about the same...

JavaScript: Well, I'm better in a couple of ways. First of all, I'm lightning fast these days. While I'm considered a scripting language, now my performance is close to that of native compiled languages.

Head First: And second?

JavaScript: My ability to do things in the browser has expanded dramatically. Using the JavaScript libraries available in all modern browsers you can find out your location, play video and audio, paint graphics on your web page and a lot more. But if you wanna do all that you have to know JavaScript.

Head First: But back to those criticisms of you, the language. I've heard some not so kind words... I believe the phrase was "hacked up language."

JavaScript: I'll stand on my record. I'm pretty much one of, if not *the* most widely used languages in the world. I've also fought off many competitors and won. Remember Java in the browser? Ha, what a joke. VBScript? Ha. JScript? Flash?! Silverlight? I could go on and on. So, tell me, how bad could I be?

Head First: You've been criticized as, well, "simplistic."

JavaScript: Honestly, it's my greatest strength. The fact that you can fire up a browser, type in a few lines of JavaScript and be off and running, that's

powerful. And it's great for beginners too. I've heard some say there's no better beginning language than JavaScript.

Head First: But simplicity comes at a cost, no?

JavaScript: Well that's the great thing, I'm simple in the sense you can get a quick start. But I'm deep and full of all the latest modern programming constructs.

Head First: Oh, like what?

JavaScript: Well, for example, can you say dynamic types, first-class functions and closures?

Head First: I can say it but I don't know what they are.

JavaScript: Figures... that's okay, if you stay with the book you will get to know them.

Head First: Well, give us the gist.

JavaScript: Let me just say this, JavaScript was built to live in a dynamic web environment, an exciting environment where users interact with a page, where data is coming in on the fly, where many types of events happen, and the language reflects that style of programming. You'll get it a little more a bit later in the book when you understand JavaScript more.

Head First: Okay, to hear you tell it, you're the perfect language. Is that right?

JavaScript tears up...

JavaScript: You know, I didn't grow up within the ivy-covered walls of academia like most languages. I was born into the real world and had to sink or swim very fast in my life. Given that, I'm not perfect; I certainly have a few "rough spots."

Head First with a slight Barbara Walters smile: We've seen a new side of you today. I think this merits another interview in the future.

BULLET POINTS

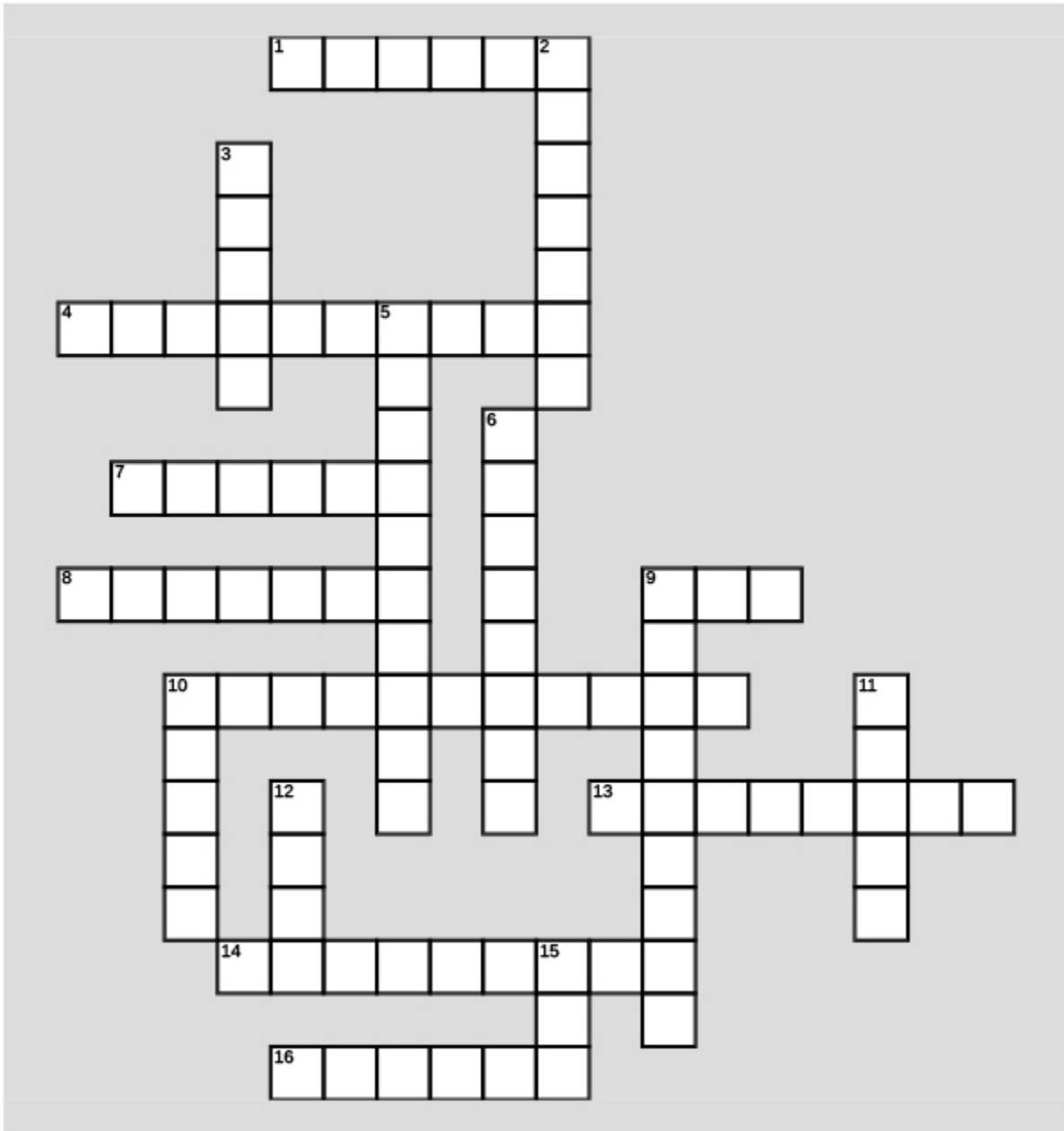
- JavaScript is used to add **behavior** to web pages.
- Browser engines are much faster at executing JavaScript than they were just a few years ago.
- Browsers begin executing JavaScript code as soon as they encounter the code in the page.
- Add JavaScript to your page with the **<script>** element.
- You can put your JavaScript inline in the web page, or link to a separate file containing your JavaScript from your HTML.
- Use the **src** attribute in the **<script>** tag to link to a separate JavaScript file.
- HTML **declares** the structure and content of your page; JavaScript **computes** values and adds behavior to your page.
- JavaScript programs are made up of a series of **statements**.
- One of the most common JavaScript statements is a variable declaration, which uses the **let** keyword to declare a new variable and the assignment operator, **=**, to assign a value to it.
- Use **const** to assign a value that shouldn't change.
- The value of a constant doesn't vary, so we call them constants, not variables.
- There are just a few rules and guidelines for naming JavaScript variables and constants, and it's important that you follow them.
- Remember to avoid JavaScript keywords when naming variables.
- JavaScript expressions compute values.

- Three common types of expressions are **numeric**, **string** and **boolean** expressions.
 - **if/else** statements allow you to make decisions in your code.
 - **while/for** statements allow you to execute code many times by looping.
 - You can group statements together into a code **block** by enclosing them in curly braces.
 - Use **console.log** instead of **alert** to display messages to the console.
 - Console messages should be used primarily for troubleshooting as users will most likely never see console messages.
 - JavaScript is most commonly found adding behavior to web pages, but is also used to script many creative applications, and is used as a server-side programming language with node. js.
-

JavaScript cross



Time to stretch your dendrites with a puzzle to help it all sink in.



ACROSS

1. Variables are used to store these.

4. Use _____ to troubleshoot your code.

7. Today's JavaScript runs a lot _____ than it used to.
8. There are 99 _____ of rootbeer on the wall.
9. To link to an external JavaScript file from HTML, you need the _____ attribute for your `<script>` element.
10. Each time through a loop, we evaluate a _____ expression.
13. The if/else statement is used to make a _____.
14. All JavaScript statements end with a _____.
16. You put your JavaScript inside a _____ element.

DOWN

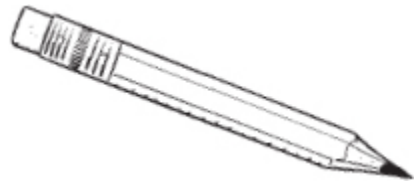
2. You can concatenate _____ together with the + operator.
3. Store values that don't change in this.
5. $3 + 4$ is an example of an _____.
6. JavaScript adds _____ to your web pages.
9. Each line of JavaScript code is called a _____.
10. To avoid embarrassing naming mistakes, use _____ case.

11. Do things more than once in a JavaScript program with the _____ loop.

12. JavaScript variable names are _____ sensitive.

15. To declare a variable, use this keyword.

SHARPEN YOUR PENCIL SOLUTION



Identify that declarations below that you think are best suited for `let` and for `const`. Remember, while all uppercase text is a convention for `const` variable, it isn't always adhered to. We've done a couple for you:

const

`DISTANCE_TO_MOON = 238900;`

↖ The distance to the moon isn't going to change during this computation, so this is a constant value. At least we hope it doesn't change!

let

`last_guess = 0;`

↖ Presumably this is to keep the value of a user guess that is going to change over time, so this is not a constant and we use let.

const

`SCREEN_WIDTH = 1024;`

↖ Although screen sizes can sometimes change, like if the browser window resizes, in this case the uppercase treatment signifies that this is intended to be a fixed constant value.

let

`counter = 100;`

↖ If we're counting something then this value needs to change over time; thus, we have a variable and use let.

const

`firstUSPresident = "Washington";`

↖ The first president of the United States will always be George Washington, so this is a const even though it doesn't use the all uppercase convention.

let

`fluxCapacitorReading = "System Normal";`

↖ You never know when the status of the flux capacitor is going to change. Sure, everything may be normal now, but watch out! So we have a variable declared with let.

BE THE BROWSER SOLUTION



Below, you'll find JavaScript code with some mistakes in it. Your job is to play like you're the browser and find the errors in the code. After you've done the exercise look at the end of the chapter to see if you found them all.

A

```
// Test for jokes
```

```
const joke = "JavaScript walked into a bar....";
```

```
let toldJoke = "false";
```

```
let $punchline =
```

```
    "Better watch out for those semi-colons."
```

```
let %entage = 20;
```

```
let result
```

```
if (toldJoke == true) {
```

```
    Alert($punchline);
```

```
} else
```

```
    alert(joke);
```

```
}
```

```
joke = "Knock, knock. Who's there? JavaScript....";
```

Delimit your strings with two double quotes (") or two single quotes ('). Don't mix!

Don't put quotes around boolean values unless you really want a string.

It's okay, but not recommended, to begin a variable with a \$.

Don't forget to end statements with a semi-colon!

Can't use % in variable names.

Another missing semi-colon.

Should be alert, not Alert. JavaScript is case-sensitive.

We're missing an opening brace here.

We're trying to change the value of a constant here.

B

```
\\ Movie Night
```

```
let zip code = 98104;
```

```
const joe'sFavoriteMovie = Forbidden Planet;
```

```
let movieTicket$ = 9;
```

```
if (movieTicket$ >= 9) {
```

```
    alert("Too much!");
```

```
} else {
```

```
    alert("We're going to see " + joe'sFavoriteMovie);
```

```
}
```

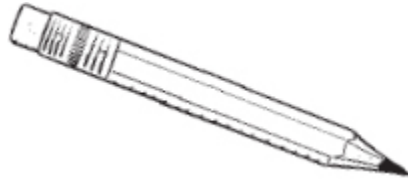
No spaces allowed in variable names.

No quotes allowed in variable names.

But we do need quotes around the string "Forbidden Planet".

This if/else doesn't work because of the invalid constant name here.

SHARPEN YOUR PENCIL SOLUTION



Get out your pencil and let's put some expressions through their paces. For each expression below, compute its value and write in your answer. Yes, **WRITE IN...** forget what your Mom told you about writing in books and scribble your answer right in this book! Here's our solution.



Can you say "Celsius to Fahrenheit calculator"?

$(9 / 5) * \text{temp} + 32$

What is the result when temp is 10? 50



This is a boolean expression. The `==` operator tests if two values are equal to each other.

`color == "orange"`

Is this expression true or false when color has the value "pink"? false
Or, has the value "orange"? true

`name + ", " + "you've won!"`

What value does this compute to when name is "Martha"?
"Martha, you've won!"

`yourLevel > 5`

This tests if the first value is greater than the second. You can also use `>=` to test if the first value is greater than or equal to the second.

When yourLevel is 2, what does this evaluate to? false

When yourLevel is 5, what does this evaluate to? false

When yourLevel is 7, what does this evaluate to? true

`(level * points) + bonus`

Okay, level is 5, points is 30,000 and bonus is 3300. What does this evaluate to? 153300

`color != "orange"`



The `!=` operator tests if two values are NOT equal to each other.

Is this expression true or false when color has the value "pink"? true

Extra CREDIT!

`1000 + "108"`

Are there a few possible answers? Only one is correct. Which would you choose? "1000108"

CODE MAGNET SOLUTION



A JavaScript program is all scrambled up on the fridge. Can you put the magnets back in the right places to make a working JavaScript program to produce the output shown below?. Here's our solution.

Here are the unscrambled magnets! ↴

```
const name = "Joe";  
let i = 0;  
while (i < 2) {  
    document.write("Happy Birthday to you.<br>");  
    i = i + 1;  
}  
document.write("Happy Birthday dear " + name + ",<br>");  
document.write("Happy Birthday to you.<br>");
```

↴ Declare a constant for the name "Joe". The name doesn't change, so we use `const`, not `let`.

↴ `i` is our while loop variable, it needs to change each time through the loop.

↴ Keep looping as long as the value in `i` is less than 2.

↴ Display a message to the browser page.

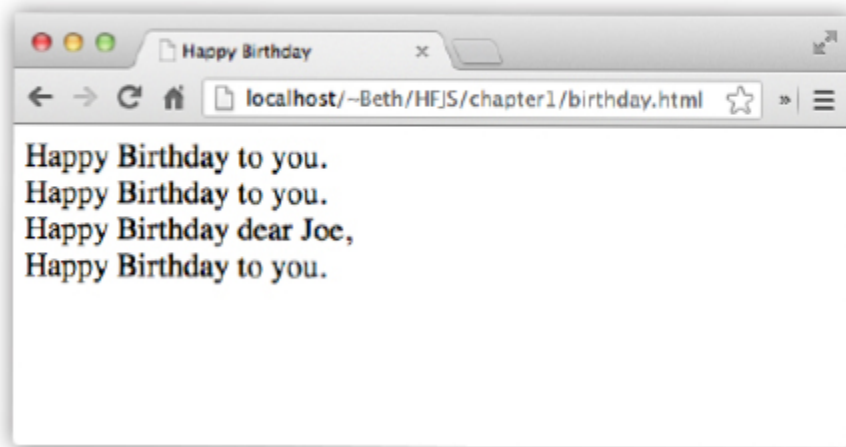
↴ Increment the while loop variable.

↴ End the while loop block.

↴ Display a message to the browser page, including the value of the variable `name`.

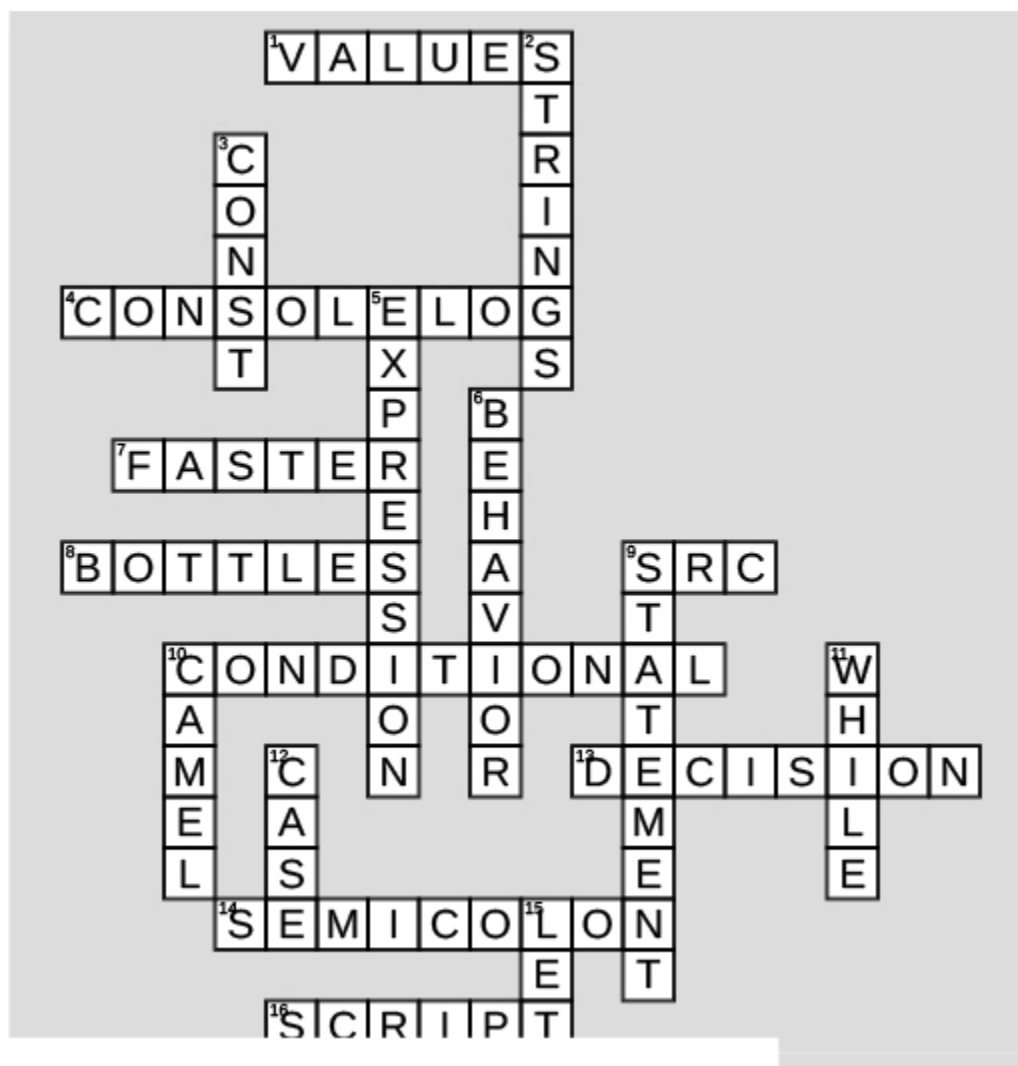
↴ Display a message to the browser page.

↓ Your unscrambled program
should produce this output.



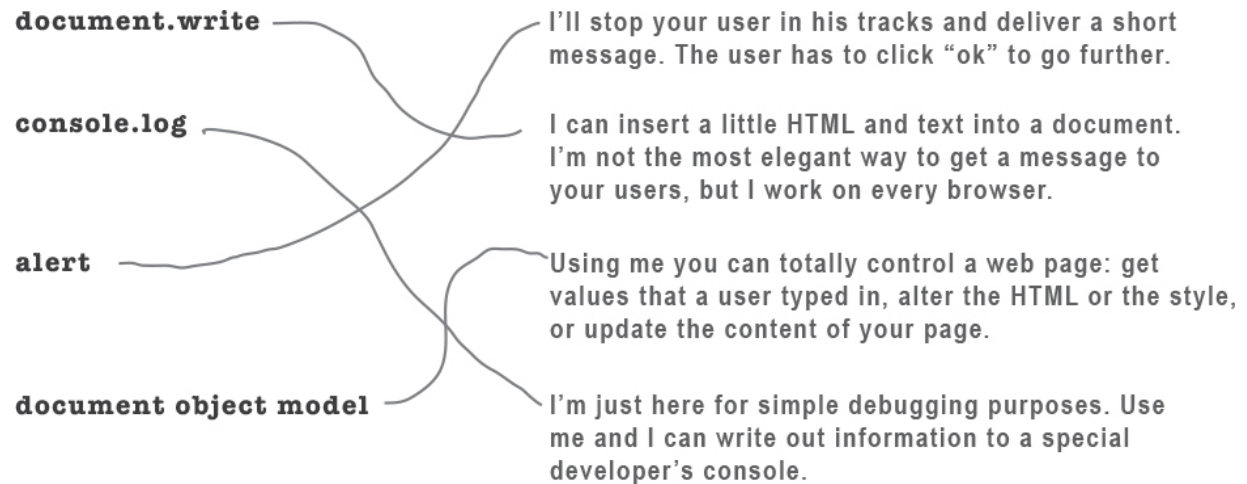
JavaScript cross Solution





WHO DOES WHAT? SOLUTION

All our methods of communication have come to the party with masks on. Can you help us unmask each one? Match the descriptions on the right to the names on the left. Here's our solution:



Chapter 2. Writing Real Code: *Going Further*

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at mcronin@oreilly.com.



You already know about variables, types, expressions... we could go on.

The point is, you already know a few things about JavaScript. In fact, you know enough to write some **real code**. Some code that does something interesting, some code that someone would want to use. What you're lacking is the **real experience** of writing code, and we're going to remedy that right here and now. How? By jumping in head first and coding up a casual game, all written in JavaScript. Our goal is ambitious but we're going to take it one step at a time. Come on, let's get this started, and if you want to launch the next startup, we won't stand in your way; the code is yours.

Let's build a Battleship game

It's you against the browser: the browser hides ships and your job is to seek them out and destroy them. Of course, unlike the real Battleship game, in this one you don't place any ships of your own. Instead, your job is to sink the computer's ships in the fewest number of guesses.

Goal: Sink the browser's ships in the fewest number of guesses. You're given a rating, based on how well you perform.

Setup: When the game program is launched, the computer places ships on a virtual grid. When that's done, the game asks for your first guess.

How you play: The browser will prompt you to enter a guess and you'll type in a grid location. In response to your guess, you'll see a result of "Hit", "Miss", or "You sank my battleship!" When you sink all the ships, the game ends by displaying your rating.

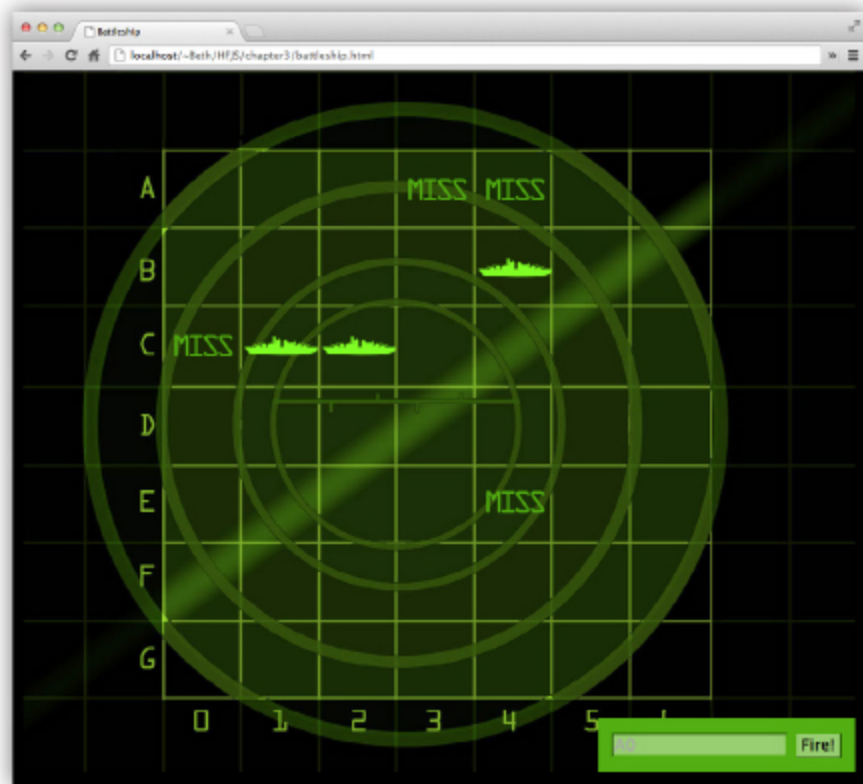
Our first attempt...

... a simplified Battleship

For our first attempt we're going to start simpler than the full-blown 7x7 graphical version with three ships. Instead we're going to start with a nice

1-D grid with seven locations and one ship to find. It will be crude, but our focus is on designing the basic code for the game, not the look and feel (at least for now).

Don't worry; by starting with a simplified version of the game, you get a big head start on building the full game later. This also gives us a nice chunk to bite off for your first real JavaScript program (not counting the Serious Business Application from [Chapter 1](#), of course). So, we'll build the simple version of the game in this chapter, and get to the deluxe version later in the book after you've learned a bit more about JavaScript.



↗ Here's what we're shooting for: a nice 7x7 grid with three ships to hunt down. Right now we're going to start a little simpler, but once you know a bit more JavaScript we'll complete the implementation so it looks just like this, complete with graphics and everything...we'll leave the sound to you as extra credit.

Instead of a 7x7 grid, like the one above,
we're going to start with just a 1x7 grid.
And, we'll worry about just one ship for now.

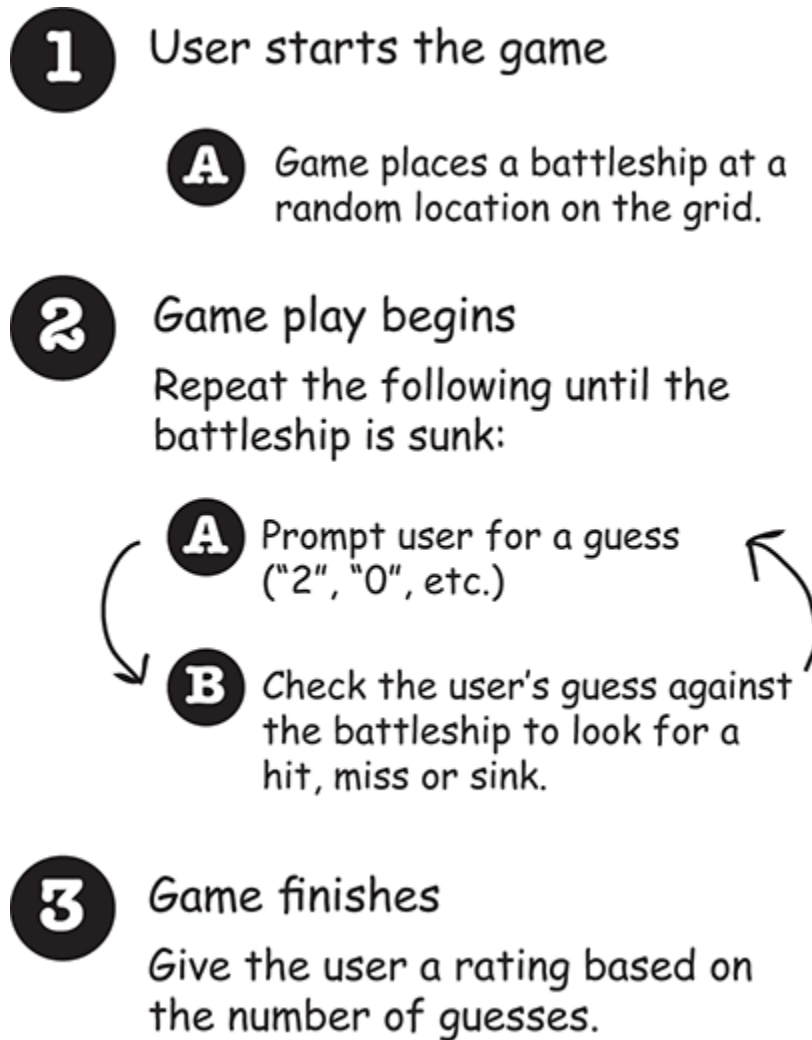


Notice that each ship
takes up three grid
locations (similar to the
real board game).

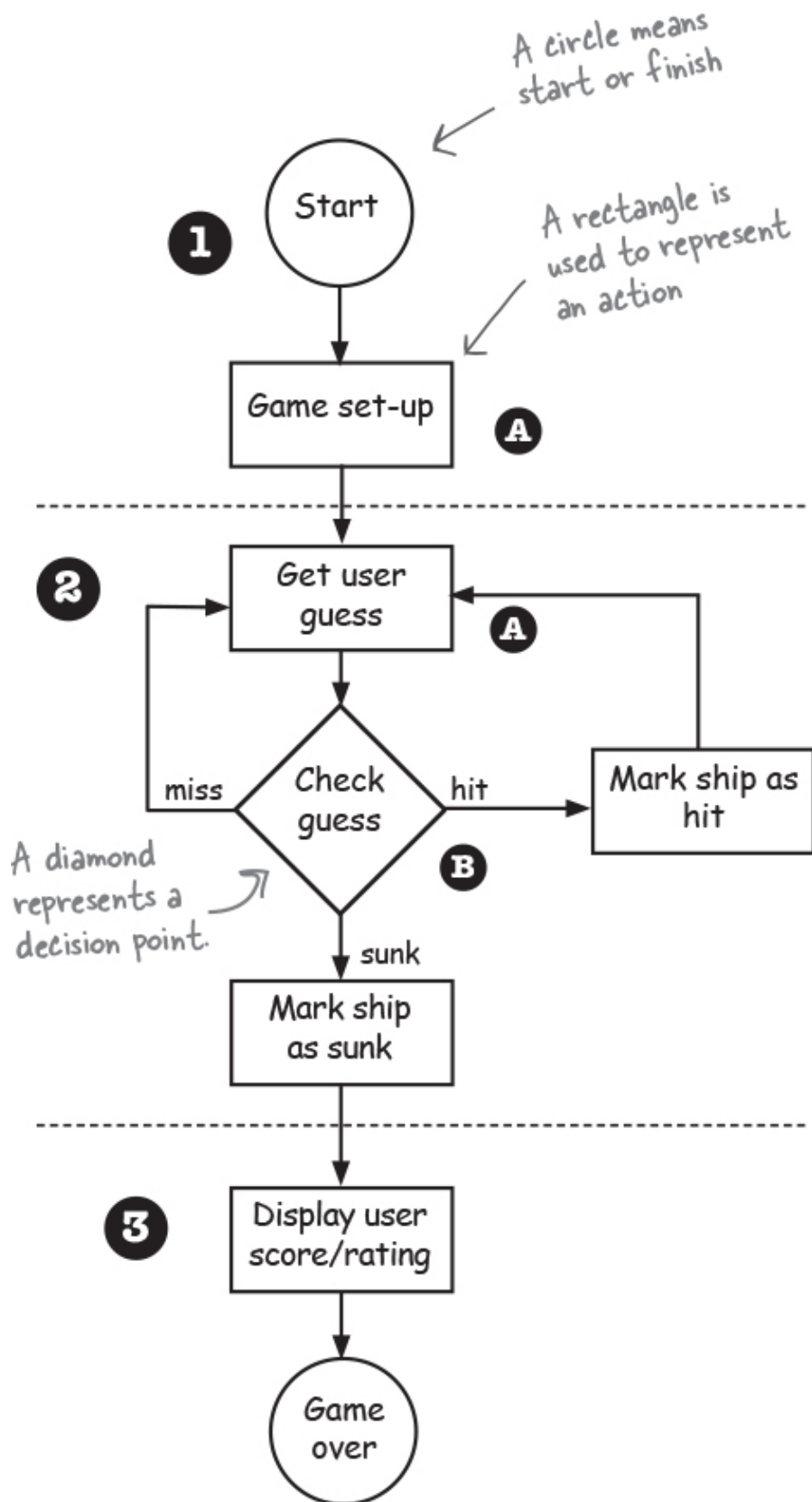
First, a high-level design

We know we'll need variables, and some numbers and strings, and if statements, and conditional tests, and loops... but where and how many? And how do we put it all together? To answer these questions, we need more information about what the game should do.

First, we need to figure out the general flow of the game. Here's the basic idea:



Now we have a high-level idea of the kinds of things the program needs to do. Next we'll figure out a few more details for the steps.



Whoa. A real flowchart.

A few more details...

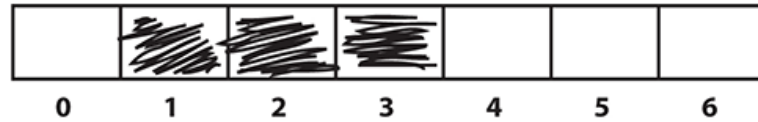
We have a pretty good idea about how this game is going to work from the high-level design and professional looking flowchart, but let's nail down just a few more of the details before we begin writing the code.

Representing the ships

For one thing, we can start by figuring out how to represent a ship in our grid. Keep in mind that the virtual grid is... well, *virtual*. In other words, it doesn't exist anywhere in the program. As long as both the game and the user know that the battleship is hidden in three consecutive cells out of a possible seven (starting at zero), the row itself doesn't have to be represented in code. You might be tempted to build something that holds all seven locations and then to try to place the ship in those locations. But, we don't need to. We just need to know the cells where the ship is located, say, at cells 1, 2 and 3.

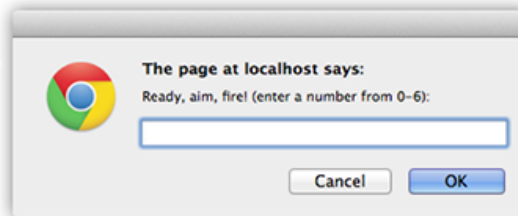
- 1** Game starts, and creates one battleship and gives it a location on three cells in the single row of seven cells.

The locations are just integers; for example, 1,2,3 are the cell locations in this picture:



- 2** Game play begins. Prompt user for a guess:

A

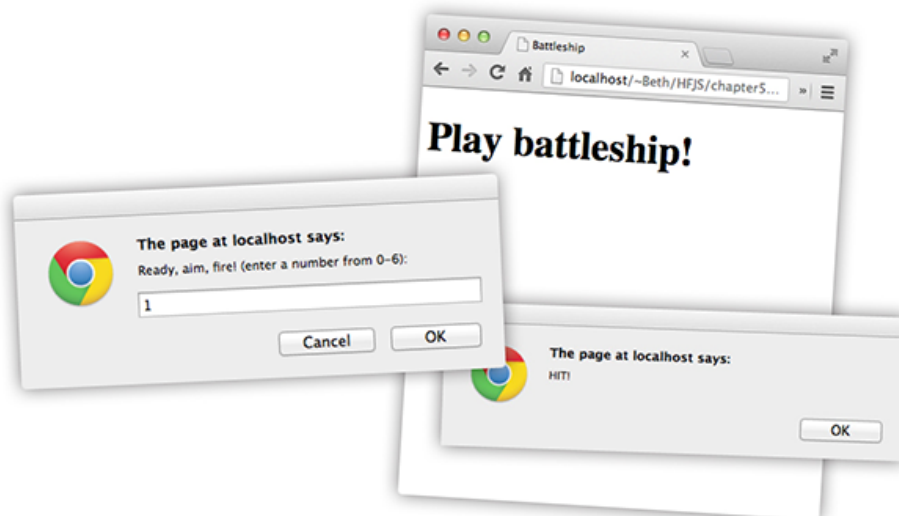


B

Check to see if user's input hit any of the ship's three cells. Keep track of how many hits there are in a variable.

- 3** Game finishes when all three cells have been hit and our number of hits variable value is 3. We tell the user how many guesses it took to sink the ship.

Sample game interaction



Getting user input

What about getting user input? We can do that with the `prompt` function. Whenever we need to get a new location from the user, we'll use `prompt` to display a message and get the input, which is just a number between 0 and 6, from the user.

Displaying the results

What about output? For now, we'll continue to use `alert` to show the output of the game. It's a bit clunky, but it'll work. (For the real game, later in the book, we'll be updating the web page instead, but we've got a way to go before we get there.)

Working through the Pseudocode

We need an approach to planning and writing our code. We're going to start by writing *pseudocode*. Pseudocode is halfway between real JavaScript code and a plain English description of the program, and as you'll see, it will help us think through how the program is going to work without fully having to develop the *real code*.

In this pseudocode for Simple Battleship, we've included a section that describes the variables we'll need, and a section describing the logic of the program. The variables will tell us what we need to keep track of in our code, and the logic describes what the code has to faithfully implement to create the game.

DECLARE three *variables* to hold the location of each cell of the ship. Let's call them location1, location2 and location3.

DECLARE a *variable* to hold the user's current guess. Let's call it guess.

DECLARE a *variable* to hold the number of hits. We'll call it hits and **set** it to 0.

DECLARE a *variable* to hold the number of guesses. We'll call it guesses and **set** it to 0.

DECLARE a *variable* to keep track of whether the ship is sunk or not. Let's call it isSunk and **set** it to false.

The variables we need.



LOOP: *while the ship is not sunk*

GET the user's guess

COMPARE the user's input to valid input values

IF the user's guess is invalid,

TELL user to enter a valid number

ELSE

ADD one to guesses

IF the user's guess matches a location

ADD one to the number of hits

IF number of hits is 3

SET isSunk to true

TELL user "You sank my battleship!"

END IF

END IF

END ELSE

END LOOP

TELL user stats

And here's the logic.



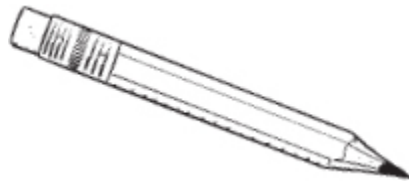
It's not JavaScript, but you can probably already see how to begin implementing this logic in code.



Notice that we're using indentation to help make the pseudocode easier to read. We'll be doing that in the real code too.



SHARPEN YOUR PENCIL



Let's say our virtual grid looks like this:



And we've represented the ship locations using our location variables, like this:

```
location1 = 3;  
location2 = 4;  
location3 = 5;
```

Use the following sequence as your test user input:

```
1, 4, 2, 3, 5
```

Now, using the pseudocode on the previous page, walk through each step of code and see how this works given the user input. Put your notes below.

We've begun the exercise for you below. If this is your first time walking through pseudocode, take your time and see how it all works.

NOTE

If you need a hint, take a quick peek at our answer at the end of the chapter.

	location1	location2	location3	guess	guesses	hits	isSunk
	3	4	5	—	0	0	false
	3	4	5	1	1	0	false

The first row shows the initial values of the variables, before the user enters their first guess. We're not initializing the variable guess, so its value is undefined.

Oh, before we go any further, don't forget the HTML!

You're not going to get very far without some HTML to link to your code. Go ahead and type the markup below into a new file named "battleship.html". After you've done that we'll get back to writing code.

```
<!doctype html>  
<html lang="en">
```

```
  <head>
```

```
    <title>Battleship</title>
```

```
    <meta charset="utf-8">
```

```
  </head>
```

```
  <body>
```

```
    <h1>Play battleship!</h1>
```

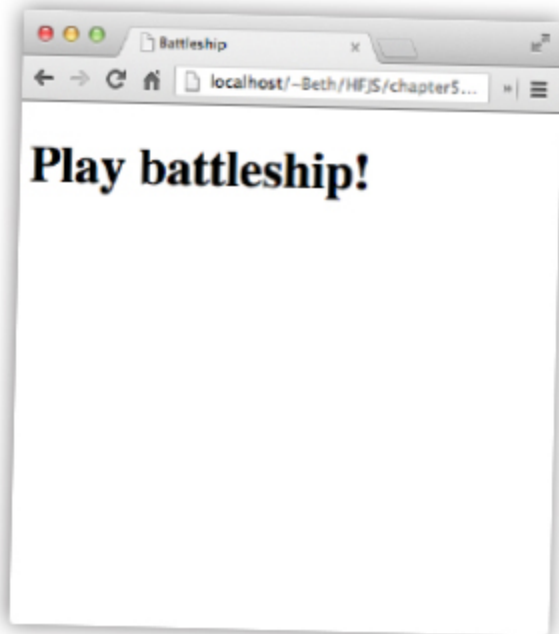
```
    <script src="battleship.js"></script>
```

```
  </body>
```

```
</html>
```

← The HTML for the Battleship game is super simple; we just need a page that links to the JavaScript code, and that's where all the action happens.

↖ We're linking to the JavaScript at the bottom of the <body> of the page, so the page is loaded by the time the browser starts executing the code in "battleship.js".



Here's what you'll see when you load the page. We need to write some code to get the game going!

BRAIN POWER



Flex those dendrites.

This is thinking ahead a bit, but what kind of code do you think it would take to generate a random location for the ship each time you load the page? What factors would you have to take into account in the code to correctly place a ship? Feel free to scribble some ideas here.

Writing the Simple Battleship code

We're going to use the pseudocode as a blueprint for our real JavaScript code. First, let's tackle all the variables we need. Take another look at our pseudocode to check out the variables we need:

DECLARE three *variables* to hold the location of each cell of the ship. Let's call them `location1`, `location2` and `location3`.

We need three variables to hold the ship's location.

DECLARE a *variable* to hold the user's current guess. Let's call it `guess`.

DECLARE a *variable* to hold the number of hits. We'll call it `hits` and **set** it to 0.

DECLARE a *variable* to hold the number of guesses. We'll call it `guesses` and **set** it to 0.

And three more (`guess`, `hits` and `guesses`) to deal with the user's guess.

DECLARE a *variable* to keep track of whether the ship is sunk or not. Let's call it `isSunk` and **set** it to `false`.

And another to track whether or not the ship is sunk.

Let's get these variables into a JavaScript file. Create a new file named "battleship.js" and type in your variable declarations like this:

```
let location1 = 3;  
let location2 = 4;  
let location3 = 5;
```

Here are our three location variables. We'll go ahead and set up a ship at locations 3, 4 and 5, just for now.

We'll come back later and write some code to generate a random location for the ship to make it harder for the user.

```
let guess;
```

```
let hits = 0;
```

```
let guesses = 0;
```

The variable `guess` won't have a value until the user makes a guess. Until then it will have the value `undefined`.

We'll assign initial values of 0 to both `hits` and `guesses`.

We didn't make these constants because we'll vary them later.

```
let isSunk = false;
```

Finally, the `isSunk` variable gets a value of `false`. We'll set this to `true` when we've sunk the ship.

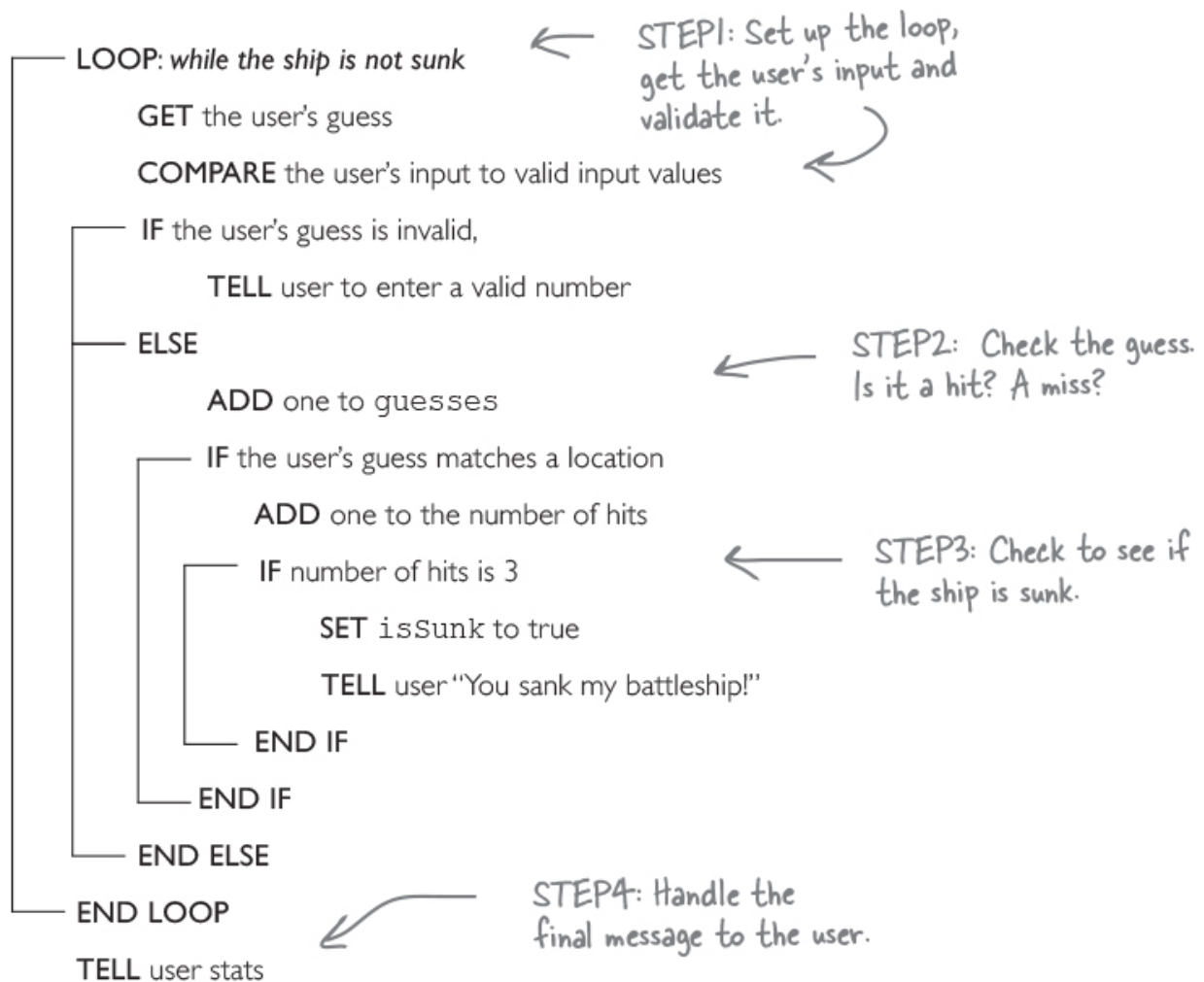
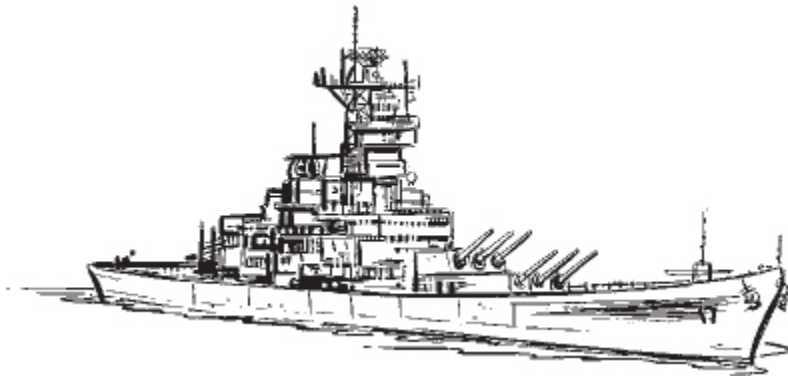
SERIOUS CODING

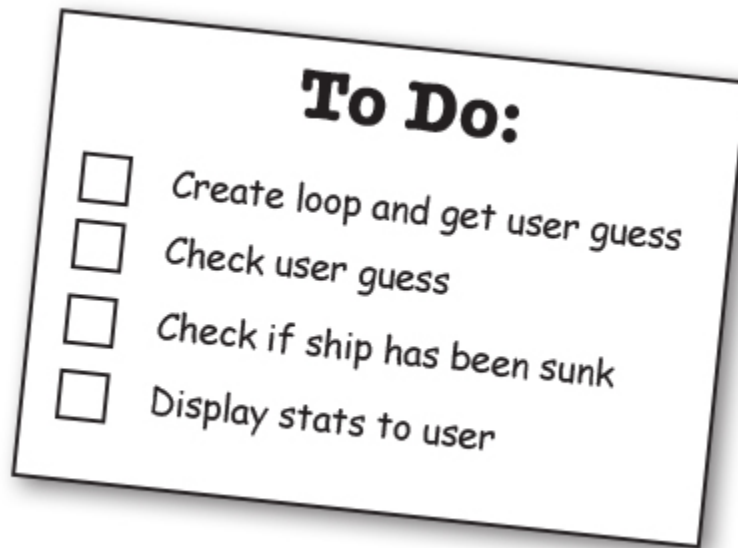


If you don't provide an initial value for a variable, then JavaScript gives it a default value of `undefined`. Think of the value `undefined` as JavaScript's way of saying "this variable hasn't been given a value yet." We'll be talking more about `undefined` and some other strange values a little later.

Now let's write the game logic

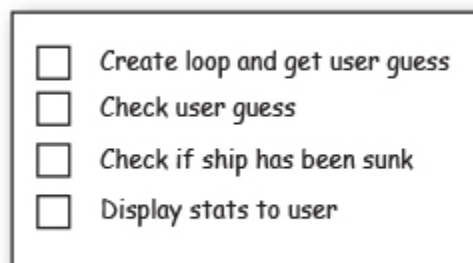
We've got the variables out of the way, so let's dig into the actual pseudocode that implements the game. We'll break this into a few pieces. The first thing you're going to want to do is implement the loop: it needs to keep looping while the ship isn't sunk. From there we'll take care of getting the guess from the user and validating it—you know, making sure it really is a number between 0 and 6— and then we'll write the logic to check for a hit on a ship and to see if the ship is sunk. Last, we'll create a little report for the user with the number of guesses it took to sink the ship.





Step One: setting up the loop, getting some input

Now we're going to begin to translate the logic of our game into actual JavaScript code. There isn't a perfect mapping from pseudocode to JavaScript, so you'll see a few adjustments here and there. The pseudocode gives us a good idea of *what* the code needs to do, and now we have to write the JavaScript code that can do the *how*.



Let's start with all the code we have so far and then we'll zero in on just the parts we're adding (to save a few trees here and there, or electrons if you're reading the digital version of the book):

DECLARE variables

```
let location1 = 3;  
let location2 = 4;  
let location3 = 5;  
let guess;  
let hits = 0;  
let guesses = 0;  
let isSunk = false;
```

← We've already covered these, but we're including them here for completeness.

Here's the start of the loop. While the ship isn't sunk, we're still in the game, so keep looping.

LOOP: while the ship is not sunk

```
while (isSunk == false) {
```

Remember, while uses a conditional test to determine whether to keep looping. In this case we're testing to make sure that isSunk is still false. We'll set it to true as soon as the ship is sunk.

GET the user's guess

```
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");
```

```
}
```

↖ Each time we go through the while loop we're going to ask the user for a guess. To do that we use the prompt built-in function. More on that on the next page...

BRAIN POWER



If you ran this code now, would the game ever end?

How prompt works

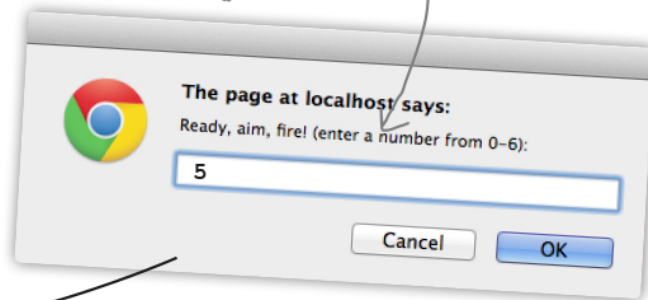
The browser provides a built-in function you can use to get input from the user, named `prompt`. The `prompt` function is a lot like the `alert` function you've already used—`prompt` causes a dialog to be displayed with a string that you provide, just like `alert`—but it also provides the user with a place to type a response. That response, in the form of a string, is then returned as a result of calling the function. Now, if the user cancels the dialog or doesn't enter anything, then `null` is returned instead.

Here we're assigning the result of the prompt function to the guess variable.

```
guess = prompt("Ready, aim, fire! (enter a number from 0-6):");
```

The prompt function's job is to get input from the user. Depending on your device, that usually happens in a dialog box.

You provide prompt with a string, which is used as instructions to your user in the dialog box.



"5"

Once the prompt function obtains input from the user, it returns that input to your code. In this case the input, in the form of a string, is assigned to the variable guess.

WATCH IT!



You might be tempted to try this code now...

...but don't. If you do, your browser will start an *infinite loop* of asking you for a guess, and then asking you for a guess, and so on, without any means of stopping the loop (other than using your operating system to force the browser process to stop).

Checking the user's guess

If you look at the pseudocode, to check the user's guess we need to first make sure the user has entered a valid input. If so, then we also check to see if the guess was a hit or miss. We'll also want to make sure we appropriately update the `guesses` and `hits` variables. Let's get started by checking the validity of the user's input, and if the input is valid, we'll increment the `guesses` variable. After that we'll write the code to see if the user has a hit or miss.

- ☒ Create loop and get user guess
- ☐ Check user guess
- ☐ Check if ship has been sunk
- ☐ Display stats to user

```
// Variable declarations go here
```

```
while (isSunk == false) {  
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");  
    if (guess < 0 || guess > 6) {  
        alert("Please enter a valid cell number!");  
    } else {  
        guesses = guesses + 1;  
    }  
}
```

We check validity by making sure the guess is between zero and six.

If the guess isn't valid, we'll tell the user with an alert.

And if the guess is valid, go ahead and add one to guesses so we can keep track of how many times the user has guessed.

Let's look a little more closely at the validity test. You know we're checking to see that the guess is between zero and six, but how exactly does this conditional test that? Let's break it down:

Try to read this like it's English: this conditional is true if the user's guess is less than zero OR the user's guess is greater than six. If either is true, then the input is invalid.

`if (guess < 0 || guess > 6) {`

This is really just two small tests put together. The first test checks if guess is less than zero.

And this one checks to see if guess is greater than six.

And this, which we call the OR operator, combines the two tests so that if either test is true, then the entire conditional is true. If both tests are false, then the statement is false, and the guess is between 0 and 6, which means it's valid.

THERE ARE NO DUMB QUESTIONS

Q: I noticed there is a cancel button on the prompt dialog box. What gets returned from the prompt function if the user hits cancel?

A: If you click cancel in the prompt dialog box then prompt returns the value null rather than a string. Remember that null means “no value”, which is appropriate in this case because you’ve cancelled without entering a value. We can use the fact that the value returned from prompt is null to check to see if the user clicked cancel, and if they did, then we could, say, end the game. We’re not doing that in our code, but keep this idea in the back of your mind as we might use it later in the book.

Q: You said that prompt always returns a string. So how can we compare a string value, like “0” or “6”, to numbers, like 0 and 6?

A: In this situation, JavaScript tries to convert the string in guess to a number in order to do the comparisons, $\text{guess} < 0$ and $\text{guess} > 6$. As long as you enter only a number, like 4, JavaScript knows how to convert the string “4” to the number 4 when it needs to. We’ll come back to the topic of type conversion in more detail later.

Q: What happens if the user enters something that isn’t a number into the prompt? Like “six” or “quit”?

A: In that case, JavaScript won’t be able to convert the string to a number for the comparison. So, you’d be comparing “six” to 6 or “quit” to 6, and

that kind of comparison will return false, which will lead to a MISS. In a more robust version of battleship, we'll check the user input more carefully and make sure they've entered a number first.

Q: With the OR operator, is it true if only one or the other is true, or can both be true?

A: Yes, both can be true. The result of the OR operator (||) is true if either of the tests is true, or if both are true. If both are false, then the result is false.

Q: Is there an AND operator?

A: Yes! The AND operator (&&) works similarly to OR, except that the result of AND is true only if both tests are true.

Q: What's an infinite loop?

A: Great question. An infinite loop is one of the many problems that plague programmers. Remember that a loop requires a conditional test, and the loop will continue as long as that conditional test is true. If your code never does anything to change things so that the conditional test is false at some point, the loop will continue forever. And ever. Until you kill your browser or reboot.

TWO-MINUTE GUIDE TO BOOLEAN OPERATORS

A boolean operator is used in a boolean expression, which results in a true or false value. There are two kinds of boolean operators: comparison operators and logical operators.

Comparison Operators

Comparison operators compare two values. Here are some common comparison operators:

<	means “less than”
-------------	-------------------

>	means “greater than”
-------------	----------------------

==	means “equal to”
-----------	------------------

===	means “exactly equal to” (we’ll come back to this one later)
------------	--

<=	means “less than or equal to”
--------------	-------------------------------

>=	means “greater than or equal to”
--------------	----------------------------------

!=	means “not equal to”
-----------	----------------------

Logical Operators

Logical operators combine two boolean expressions to create one boolean result (true or false). Here are two logical operators:

||

means OR. Results in true if *either* of the two expressions is true.

&&

means AND. Results in true if *both* of the two expressions are true.

Another logical operator is NOT, which acts on one boolean expression (rather than two):

!

means NOT. Results in true if the expression evaluates to false.

So, do we have a hit?

This is where things get interesting—the user’s taken a guess at the ship’s location and we need to write the code to determine if that guess has hit the ship. More specifically, we need to see if the guess matches one of the locations of the ship. If it does, then we’ll increment the `hits` variable.

- ☒ Create loop and get user guess
- ☐ Check user guess
- ☐ Check if ship has been sunk
- ☐ Display stats to user

Here's a first stab at writing the code for the hit detection; let's step through it:

```
if (guess == location1) {  
    hits = hits + 1;  
} else if (guess == location2) {  
    hits = hits + 1;  
} else if (guess == location3) {  
    hits = hits + 1;  
}
```

← If the guess is at location1, then we hit the ship, so increment the hits variable by one.

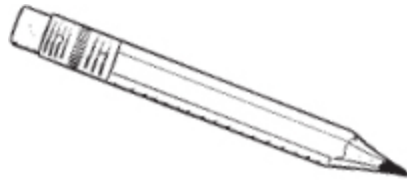
← Otherwise, if the guess is location2, then do the same thing.

← Finally, if the guess is location3, then we need to increment the hits variable.

↑ And if none of these are true, then the hits variable is never incremented.

← Notice we're using indentation for the code in each if/else block. This makes your code easier to read, especially when you've got lots of blocks nested inside blocks.

SHARPEN YOUR PENCIL



What do you think of this first attempt to write the code to detect when a ship is hit? Does it look more complex than it needs to be? Are we repeating code in a way that seems a bit, well, redundant? Could we simplify it? Using what you know of the `||` operator (that is, the boolean OR operator), can you simplify this code? *Make sure you check your answer at the end of the chapter before moving on.*

Adding the hit detection code

Let's put everything together from the previous couple of pages:

☒	Create loop and get user guess
☒	Check user guess
☐	Check if ship has been sunk
☐	Display stats to user

```

// Variable declarations go here

LOOP: while the ship is not sunk
GET the user's guess
ADD one to guesses

IF the user's guess matches a location
ADD one to the number of hits

while (isSunk == false) {
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");
    if (guess < 0 || guess > 6) {
        alert("Please enter a valid cell number!");
    } else {
        guesses = guesses + 1;
        if (guess == location1 || guess == location2 || guess == location3) {
            hits = hits + 1;
        }
    }
}

```

← Check the user's guess...

The user's guess looks valid, so let's increase the number of guesses by one.

If the guess matches one of the ship's locations we increment the hits counter.

We've combined the three conditionals into one if statement using || (OR). So read it like this: "If guess is equal to location1 OR guess is equal to location2 OR guess is equal to location3, increment hits."

Hey, you sank my battleship!

☒	Create loop and get user guess
☒	Check user guess
☒	Check if ship has been sunk
☐	Display stats to user

We're almost there; we've almost got this game logic nailed down. Looking at the pseudocode again, what we need to do now is test to see if we have three hits. If we do, then we've sunk a battleship. And, if we've sunk a battleship then we need to set `isSunk` to true and also tell the user they've destroyed a ship. Let's sketch out the code again before adding it in:

```

if (hits == 3) {
    isSunk = true;
    alert("You sank my battleship!");
}

```

First check to see if there are three hits.

Take another look at the while loop above. What happens when isSunk is true?

And if so, set isSunk to true.

And also let the user know.

Provide some post-game analysis

After `isSunk` is set to true, the while loop is going to stop looping. That's right, this program we've come to know so well is going to stop executing the body of the while loop, and before you know it the game's going to be over. But, we still owe the user some stats on how they did. Here's some code that does that:

- ☒ Create loop and get user guess
- ☒ Check user guess
- ☒ Check if ship has been sunk
- ☒ Display stats to user

```

let stats = "You took " + guesses + " guesses to sink the battleship, " +
    "which means your shooting accuracy was " + (3/guesses);
alert(stats);

```

Here we're creating a string that contains a message to the user including the number of guesses they took, along with the accuracy of their shots. Notice that we're splitting up the string into pieces (to insert the variable `guesses`, and also to fit the string into multiple lines) using the concatenation operator, `+`. For now just type this as is, and we'll explain more about this later.

Now let's add this and the sunk ship detection into the rest of the code:

```
// Variable declarations go here

LOOP: while the ship is not sunk
GET the user's guess
    while (isSunk == false) {
        guess = prompt("Ready, aim, fire! (enter a number from 0-6):");
        if (guess < 0 || guess > 6) {
            alert("Please enter a valid cell number!");
        } else {
            ADD one to guesses
                guesses = guesses + 1;

            IF the user's guess matches a location
                ADD one to the number of hits
                    if (guess == location1 || guess == location2 || guess == location3) {
                        hits = hits + 1;

                        IF number of hits is 3
                            SET isSunk to true
                                isSunk = true;

                            TELL user "You sank my battleship!"
                                alert("You sank my battleship!");
                    }
                }
            }
        }

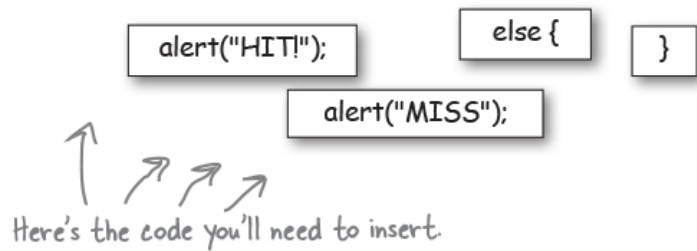
    }

TELL user stats
    let stats = "You took " + guesses + " guesses to sink the battleship, " +
        "which means your shooting accuracy was " + (3/guesses);
    alert(stats);
```

EXERCISE



Remember we said pseudocode often isn't perfect? Well we actually left something out of our original pseudocode: we're not telling the user if her guess is a HIT or a MISS. Can you insert these pieces of code in the proper place to correct this?



```
// Variable declarations go here

while (isSunk == false) {
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");
    if (guess < 0 || guess > 6) {
        alert("Please enter a valid cell number!");
    } else {
        guesses = guesses + 1;
        if (guess == location1 || guess == location2 || guess == location3) {
            hits = hits + 1;
            if (hits == 3) {
                isSunk = true;
                alert("You sank my battleship!");
            }
        }
    }
}

let stats = "You took " + guesses + " guesses to sink the battleship, " +
    "which means your shooting accuracy was " + (3/guesses);

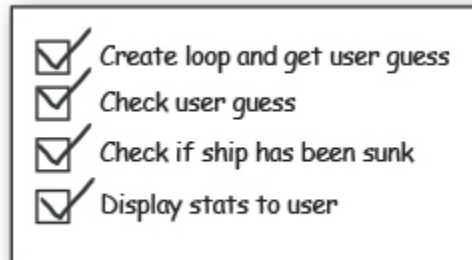
alert(stats);
```

Handwritten note: "This is a lot of curly braces to match. If you're having trouble matching them, just draw lines right in the book to match them up." with an arrow pointing to the closing brace of the while loop.

And that completes the logic!

Alright! We've now fully translated the pseudocode to actual JavaScript code. We even discovered something we left out of the pseudocode and

we've got that accounted for too. Below you'll find the code in its entirety. Make sure you have this typed in and saved in "battleship.js":



```
let location1 = 3;
let location2 = 4;
let location3 = 5;
let guess;
let hits = 0;
let guesses = 0;
let isSunk = false;

while (isSunk == false) {
    guess = prompt("Ready, aim, fire! (enter a number 1-6)");
    if (guess < 0 || guess > 6) {
        alert("Please enter a valid cell number!");
    } else {
        guesses = guesses + 1;

        if (guess == location1 || guess == location2 || guess == location3) {
            hits++;
            if (hits == 3) {
                isSunk = true;
            }
        }
    }
}
```

```
        alert("HIT!");
        hits = hits + 1;
        if (hits == 3) {
            isSunk = true;
            alert("You sank my battleship!");
        }
    } else {
        alert("MISS");
    }
}
}
let stats = "You took " + guesses + " guesses to
            "which means your shooting accuracy is " +
            (100 / guesses) + "%";
alert(stats);
```

Doing a little Quality Assurance

QA, or quality assurance, is the process of testing software to find defects. So we're going to do a little QA on this code. When you're ready, load "battleship.html" in your browser and start playing. Try some different things. Is it working perfectly? Or did you find some issues? If so list them here. You can see our test run on this page too.

QA Notes



Jot down anything that doesn't work the way it should, or that could be improved.



Here's what our game interaction looked like.





Boolean operators allow you to write more complex statements of logic.

You've seen enough conditionals to know how to test, say, if the temperature is greater than 32 degrees. Or, that a variable that represents

whether an item is inStock is true. But sometimes we need to test more. Sometimes we need to know not only if a value is greater than 32, but also if it's less than 100. Or, if an item is inStock, and also onSale. Or that an item is on sale only on Tuesdays when the user is a VIP member. So, you see, these conditionals can get complex.

Let's step through a few to get a better idea of how they work.

Say we need to test that an item is inStock AND onSale. We could do that like this:

```
if (inStock == true) {  
  if (onSale == true) {  
    // sounds like a bargain!  
    alert("buy buy buy!");  
  }  
}
```

First, see if the item is in stock...

And, if so, then see if it is on sale.

And if so, then take some action, like buy a few!

Notice this code is executed only if both conditionals are true!

We can simplify this code by combining these two conditionals together. Unlike in Simple Battleship, where we tested if `guess < 0` OR `guess > 6`, here we want to know if inStock is true AND onSale is true. Let's see how to do that...

Here's our AND operator. With AND this combined conditional is true only if the first part AND the second part are true.

↓

```
if (inStock == true && onSale == true) {  
    // sounds like a bargain!  
    alert("buy buy buy!");  
}
```

↖ Not only is this code more concise, it's also more readable. Compare this code with the code on the previous page to see.

We don't have to stop there; we can use multiple boolean operators to combine conditionals in a variety of ways:

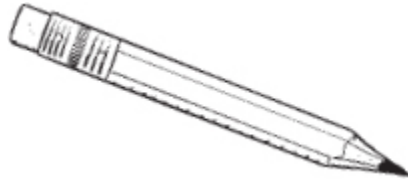
Now we're using both AND and OR in the same conditional expression. This one says: If an item is in stock AND it's either on sale, OR the price is less than 60, then buy.

↖

```
if (inStock == true && (onSale == true || price < 60)) {  
    // sounds like a bargain!  
    alert("buy buy buy!");  
}
```

↑
Notice we're using parentheses to group the conditionals together so we get the result of the OR first, and then use that result to compute the result of the AND.

SHARPEN YOUR PENCIL



We've got a whole bunch of boolean expressions that need evaluating below. Fill in the blanks, and then check your answers at the end of the chapter before you go on.

```
let temp = 81;
let willRain = true;
let humid = (temp > 80 && willRain == true);
```

What's the value of **humid**? _____

```
let guess = 6;
let isValid = (guess >= 0 && guess <= 6);
```

What's the value of **isValid**? _____

```
let kB = 1287;
let tooBig = (kB > 1000);
let urgent = true;
let sendFile =
  (urgent == true || tooBig == false);
```

What's the value of **sendFile**? _____

```
let keyPressed = "N";
let points = 142;
let level;
if (keyPressed == "Y" ||
    (points > 100 && points < 200)) {
  level = 2;
} else {
  level = 1;
}
```

What's the value of **level**? _____

EXERCISE



Bob and Bill, both from accounting, are working on a new price checker application for their company's web site. They've both written if/else statements using boolean expressions. Both are sure they've written the correct code. Which accountant is right? Should these accountants even be writing code? Check your answer at the end of the chapter before you go on.

Bob's solution

```
↪ if (price < 200 || price > 600) {  
    alert("Price is too low or too high! Don't buy the gadget.");  
} else {  
    alert("Price is right! Buy the gadget.");  
}
```

Who got it right? Bob or Bill?

Bill's solution

```
↪ if (price >= 200 || price <= 600) {  
    alert("Price is right! Buy the gadget.");  
} else {  
    alert("Price is too low or too high! Don't buy the gadget.");  
}
```



Can we talk about your verbosity...

We don't know how to bring this up, but you've been a little verbose in specifying your conditionals. What do we mean? Take this condition for instance:

We often compare our boolean variables to true or false to form our conditional.

```
if (inStock == true) {  
    ...  
}
```

And, inStock is a variable that holds a boolean value of true or false.

As it turns out, that's a bit of overkill. The whole point of a conditional is that it evaluates to either true or false, but our boolean variable `inStock` already is one of those values. So, we don't need to compare the variable to anything; it can just stand on its own. That is, we can just write this instead:

```
if (inStock) {  
    ...  
}
```

If we just use the boolean variable by itself, then if that variable is true, the conditional test is true, and the block is executed.

And if inStock is false, then the conditional test fails and the code block is skipped.

Now, while some might claim our original, verbose version was clearer in its intent, it's more common to see the more succinct version in practice. And, you'll find the less verbose version easier to read as well.

EXERCISE



We've got two statements below that use the `onSale` and `inStock` variables in conditionals to figure out the value of the variable `buyIt`. Work through each possible value of `inStock` and `onSale` for both statements. Which version is the biggest spender?

```
let buyIt = (inStock || onSale);
```

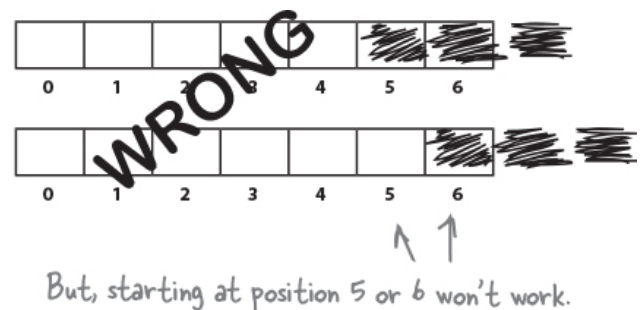
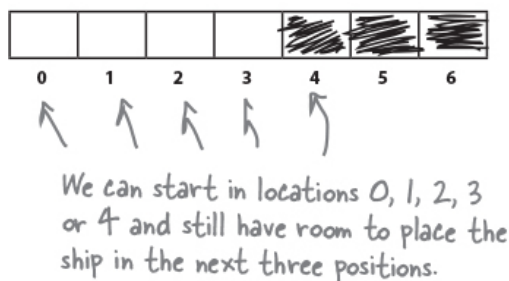
onSale	inStock	buyIt	buyIt
true	true		
true	false		
false	true		
false	false		

```
let buyIt = (inStock && onSale);
```

Finishing the Simple Battleship game

Yes, we still have one little matter to take care of because right now you've hard coded the location of the ship—no matter how many times you play the game, the ship is always at locations 3, 4 and 5. That actually works out well for testing, but we really need to randomly place the ship to make it a little more interesting to the user.

Let's step back and think about the right way to place a ship on the 1-D grid of seven cells. We need a starting location that allows us to place three consecutive positions on the grid. That means we need a starting location from zero to four.



How to assign random locations

Now, once we have a starting location (between zero and four), we simply use it and the following two locations to hold the ship.

NOTE

Take the random location along with the next two consecutive locations.

```
let location1 = randomLoc;  
let location2 = location1 + 1;  
let location3 = location2 + 1;
```

Okay, but how do we generate a random number? That's where we turn to JavaScript and its built-in functions. More specifically, JavaScript comes with a bunch of built-in math-related functions, including a couple that can be used to generate random numbers. Now we're going to get deeper into built-in functions, and functions in general, a little later in the book. For now, we're just going to make use of these functions to get our job done.

The recipe for generating a random number

We're going to start with the `Math.random` function. By calling this function we'll get back a random decimal number:

Our variable `randomLoc`. We want to assign a number from 0 to 4 to this variable.

`Math.random` is part of standard JavaScript and returns a random number.

↪ `let randomLoc = Math.random();`

↙
The only problem is it returns numbers like 0.128, 0.830, 0.9, 0.42. These numbers are between 0 and 1 (not including exactly 1). So we need a way to use this to generate random numbers 0-4.

What we need is an integer between 0 and 4—that is, 0, 1, 2, 3 or 4—not a decimal number, like 0.34. To start, we could multiply the number returned by `Math.random` by 5 to get a little closer; here's what we mean...

First, if we multiply the random number by 5, then we get a number between 0 and 5, but not including 5. Like 0.13983, 4.231, 2.3451, or say 4.999.

`let randomLoc = Math.random() * 5;`

↖ ↙
Remember, `*` means multiplication.

That's closer! Now all we need to do is clip off the end of the number to give us an integer number. To do that we can use another built-in Math function, `Math.floor`:

We can use `Math.floor` to round down all these numbers to their nearest integer value.


`let randomLoc = Math.floor(Math.random() * 5);`


So, for instance, 0.13983 becomes 0,
2.34 becomes 2 and 4.999 becomes 4.

THERE ARE NO DUMB QUESTIONS

Q: If we're trying to generate a number between 0 and 4, why does the code have a 5 in it, as in

```
Math.floor(Math.random() * 5)?
```

A: Good question. First, `Math.random` generates a number between 0 and 1, but not including 1. The maximum number you can get from `Math.random` is 0.999.... When you multiply that number by 5, the highest number you'll get is 4.999... `Math.floor` always rounds a number down, so 1.2 becomes 1, but so does 1.9999. If we generate a number from 0 to 4.999... then everything will be rounded down to 0 to 4. This is not the only way to do it, and in other languages it's often done differently, but this is how you'll see it done in most JavaScript code.

Q: So if I wanted a random number between 0 and 100 (including 100), I'd write

```
Math.floor(Math.random() * 101)?
```

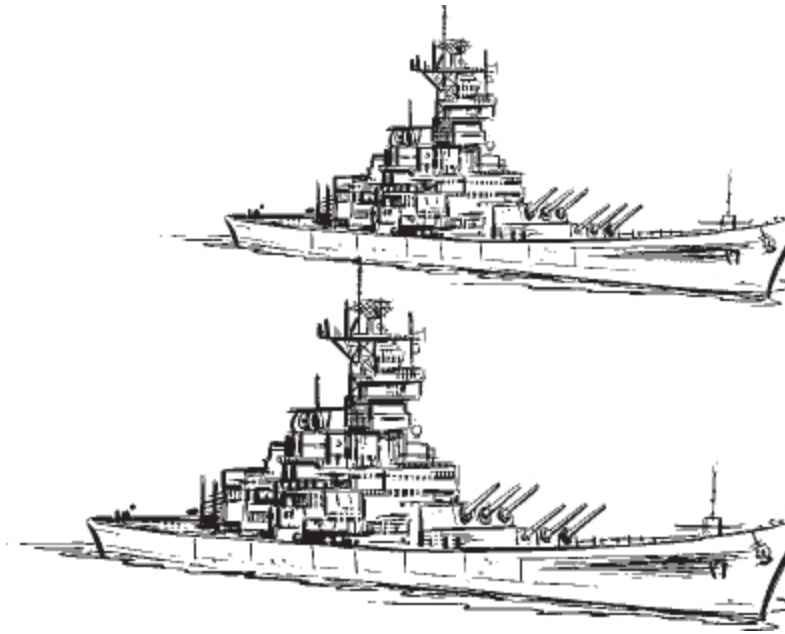
A: That's right! Multiplying by 101, and using `Math.floor` to round down, ensures that your result will be at most 100.

Q: What are the parentheses for in `Math.random()`?

A: We use parentheses whenever we “call” a function. Sometimes we need to hand a value to a function, like we do when we use `alert` to display a message, and sometimes we don’t, like when we use `Math.random`. But whenever you’re calling a function (whether it’s built-in or not), you’ll need to use parentheses. Don’t worry about this right now; we’ll get into all these details in the next chapter.

Q: I can’t get my battleship game to work. I’m not seeing anything in my web page except the “Play battleship” heading. How can I figure out what I did wrong?

A: This is where using the console can come in handy. If you’ve made an error like forgetting a quote on a string, then JavaScript will typically complain about the syntax of your program not being right, and may even show you the line number where your error is. Sometimes errors are more subtle, however. For instance, if you mistakenly write `isSunk = false` instead of `isSunk == false`, you won’t see a JavaScript error, but your code won’t behave as you expect it to. For this kind of error, try using `console.log` to display the values of your variables at various points in your code to see if you can track down the error.

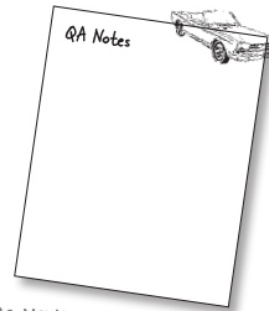


Back to do a little more QA

That's all we need. Let's put this code together (we've already done that below) and replace your existing location code with it. When you're finished, give it a few test runs to see how fast you can sink the enemy.

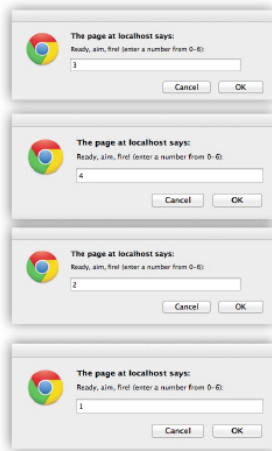
```
let randomLoc = Math.floor(Math.random() * 5);  
let location1 = randomLoc;  
let location2 = location1 + 1;  
let location3 = location2 + 1;  
let guess;  
let hits = 0;  
let guesses = 0;  
let isSunk = false;
```

```
while (isSunk == false) {  
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");  
    if (guess < 0 || guess > 6) {  
        // the rest of your code goes here....  
    }  
}
```



Go ahead and replace your location variable declarations with these new statements.

Here's one of our test sessions. The game's a little more interesting now that we've got random locations for the ship. But we still managed to get a pretty good score...

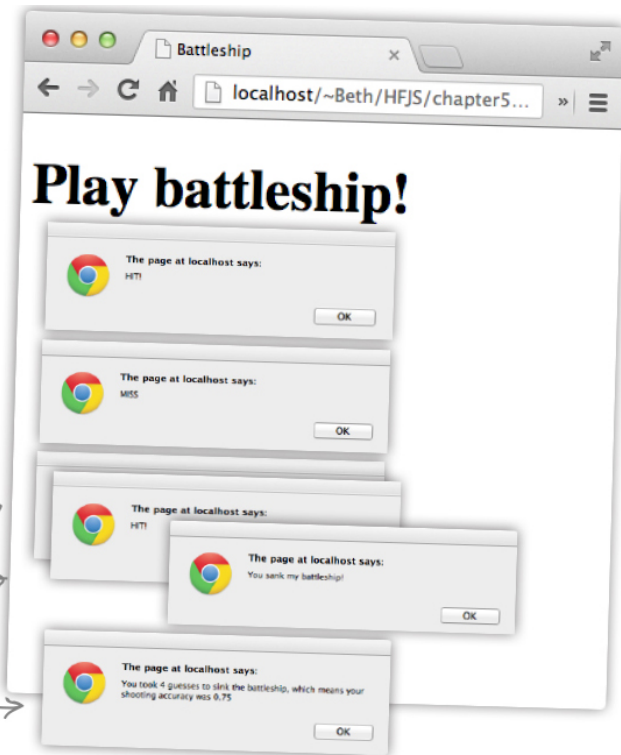


← We get a hit on our first guess. →

← On our second guess, we miss. →

← But then we get two hits in a row. →

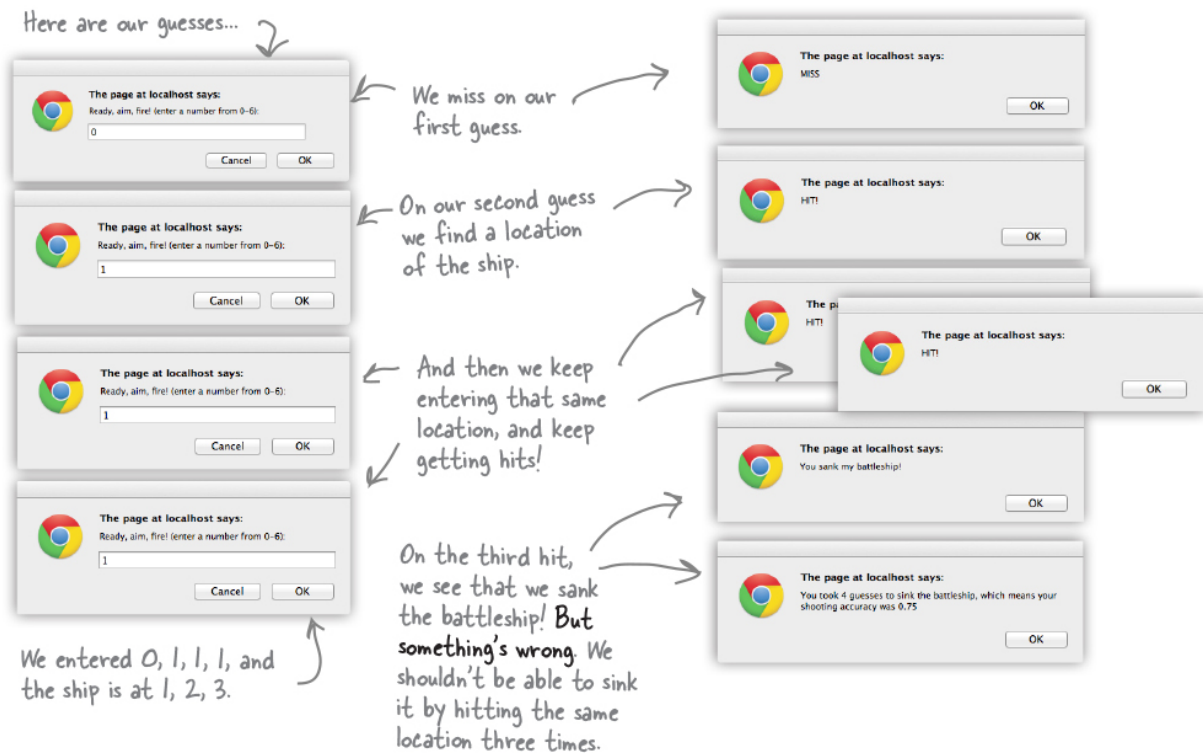
On the last hit, we sink the battleship! →



EXERCISE



Wait a sec, we noticed something that looks wrong. Hint: when we enter 0, 1, 1, 1 things don't look right! Can you figure out what's happening?



It's a cliff-hanger!

Will we **find** the bug?

Will we **fix** the bug?

Stay tuned for a much improved version of Battleship a little later in the book...

And in the meantime, see if you can come up with ideas for how you might fix the bug.

QA NOTES



Found a bug! Entering the same number that is a hit on a ship results in sinking the ship, when it shouldn't.

Congrats on your first true JavaScript program, and a short word about reusing code

You've probably noticed that we made use of a few *built-in functions* like `alert`, `prompt`, `console.log` and `Math.random`. With very

little effort, these functions have given you the ability to pop up dialog boxes, log output to the console and generate random numbers, almost like magic. But, these built-in functions are just packaged up code that's already been written for you, and as you can see their power is that you can use and reuse them just by making a call to them when you need them.

Now there's a lot to learn about functions, how to call them, what kinds of values you can pass them, and so on, and we're going to start getting into all that in the next chapter where you learn to create your own functions.

But before you get there you've got the bullet points to review, a crossword puzzle to complete... oh, and a good night's sleep to let everything sink in.

BULLET POINTS

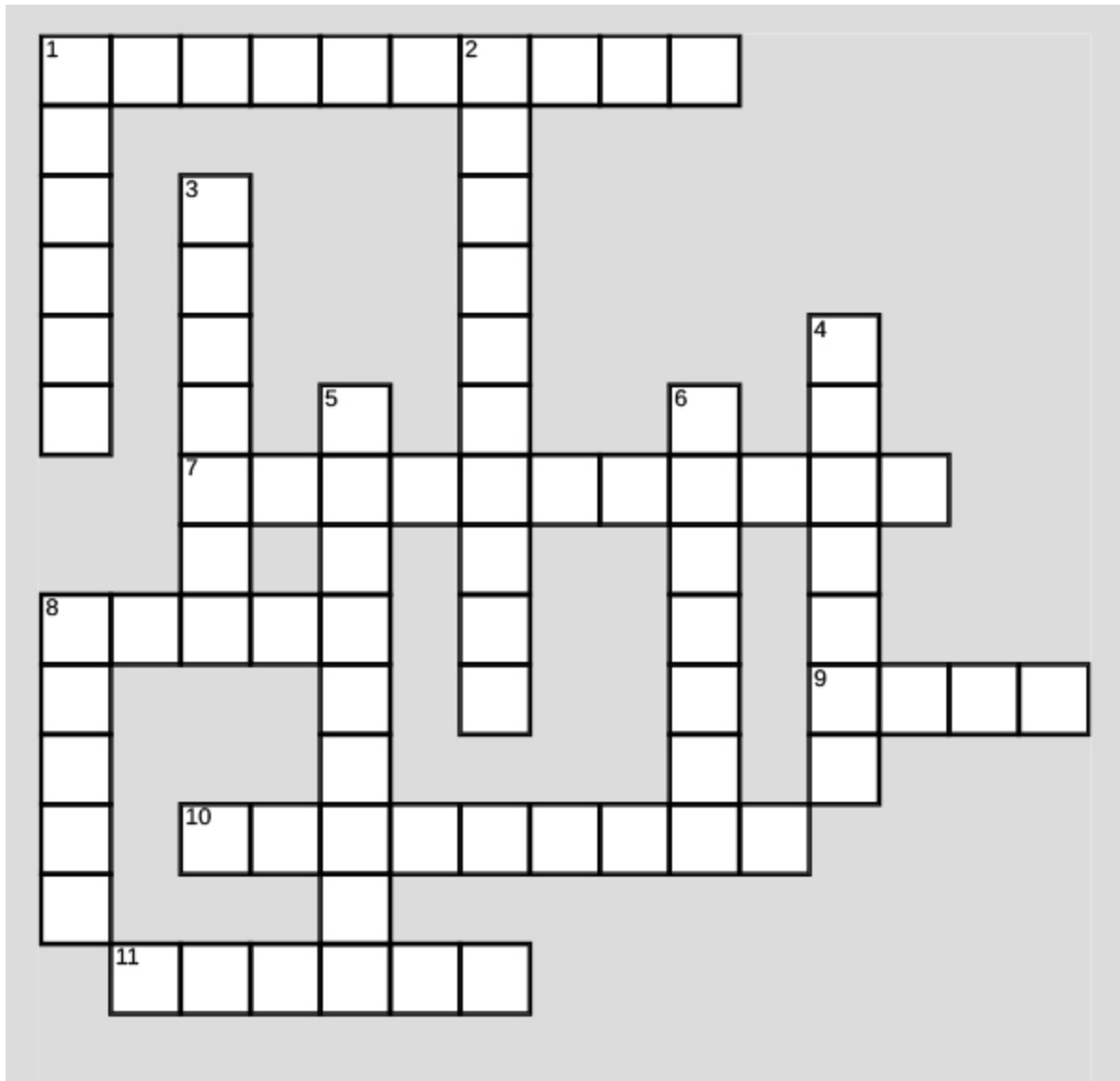
- You can use a flowchart to outline the logic of a JavaScript program, showing decision points and actions.
- Before you begin writing a program, it's a good idea to sketch out what your program needs to do with pseudocode.
- **Pseudocode** is an approximation of what your real code should do.
- There are two kinds of boolean operators: comparison operators and logical operators. When used in an expression, boolean operators result in a true or false value.
- **Comparison** operators compare two values and result in true or false. For example, we can use the boolean comparison operator `<` (“less than”) like this: `3 < 6`. This expression results in true.
- **Logical** operators combine two boolean values. For example `true || false` results in true; `true && false` results in false.
- You can generate a random number between 0 and 1 (including 0, but not including 1) using the **Math.random** function.
- The **Math.floor** function rounds down a decimal number to the nearest integer.
- Make sure you use Math with an uppercase M, and not m, when using `Math.random` and `Math.floor`.
- The JavaScript function **prompt** shows a dialog with message and a space for the user to enter a value.

- In this chapter, we used prompt to get input from the user, and alert to display the results of the battleship game in the browser.
-

JavaScript cross



How does a crossword puzzle help you learn JavaScript? The mental twists and turns burn the JavaScript right into your brain!



ACROSS

1. This helps you think about how a program is going to work.
7. Both while and if statements use _____ tests.
8. Boolean operators always result in true or _____.

9. To get a true value from an AND operator (&&), both parts of the conditional must be _____.

10. JavaScript has many built-in _____ like alert and prompt.

11. To randomly choose a position for a ship, use Math._____.

DOWN

1. To get input from a user, you can use the _____ function.

2. == is a _____ operator you can use to test to see if two values are the same.

3. OR (||) and AND (&&) are _____ operators.

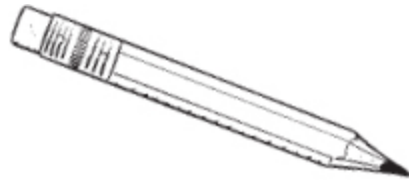
4. If you're good at testing programs, you might want to become a _____ Assurance specialist.

5. If you don't initialize a variable, the value is _____.

6. We keep track of whether a ship is sunk or not with a _____ variable.

8. To get a false value from an OR operator (||), both parts of the conditional must be _____.

SHARPEN YOUR PENCIL SOLUTION



Let's say our virtual row looks like this:



And we've represented that by setting:

```
location1 = 3;  
location2 = 4;  
location3 = 5;
```

Assume the following user input:

```
1, 4, 2, 3, 5
```

Now, using the pseudocode on the previous page, trace through each step of code, and see how this works. Put your notes below. We've started the trace for you below. Here's our solution.

location1	location2	location 3	guess	guesses	hits	isSunk
3	4	5	—	0	0	false
3	4	5	1	1	0	false
3	4	5	4	2	1	false
3	4	5	2	3	1	false
3	4	5	3	4	2	false
3	4	5	5	5	3	true

EXERCISE SOLUTION



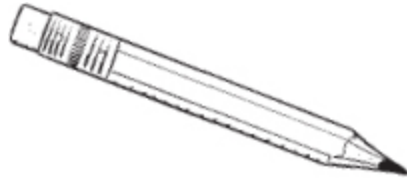
We've got two statements below that use the `onSale` and `inStock` variables in conditionals to figure out the value of the variable `buyIt`. Work through each possible value of `inStock` and `onSale` for both statements. Which version is the biggest spender? The OR (`||`) operator!

```
let buyIt = (inStock || onSale);
```

onSale	inStock	buyIt	buyIt
true	true	true	true
true	false	true	false
false	true	true	false
false	false	false	false

```
let buyIt = (inStock && onSale);
```

SHARPEN YOUR PENCIL SOLUTION



We've got a whole bunch of boolean expressions that need evaluating below. Fill in the blanks. Here's our solution:

```
let temp = 81;
let willRain = true;
let humid = (temp > 80 && willRain == true);
```

What's the value of **humid**? true

```
let guess = 6;
let isValid = (guess >= 0 && guess <= 6);
```

What's the value of **isValid**? true

```
let kB = 1287;
let tooBig = (kB > 1000);
let urgent = true;
let sendFile =
  (urgent == true || tooBig == false);
```

What's the value of **sendFile**? true

```
let keyPressed = "N";
let points = 142;
let level;
if (keyPressed == "Y" ||
    (points > 100 && points < 200)) {
  level = 2;
} else {
  level = 1;
}
```

What's the value of **level**? 2

EXERCISE SOLUTION



Bob and Bill, both from accounting, are working on a new price checker application for their company's web site. They've both written if/else statements using boolean expressions. Both are sure they've written the correct code. Which accountant is right? Should these accountants even be writing code? Here's our solution.

Bob's solution



```
if (price < 200 || price > 600) {  
    alert("Price is too low or too high! Don't buy the gadget.");  
} else {  
    alert("Price is right! Buy the gadget.");  
}
```

Bill's solution



```
if (price >= 200 || price <= 600) {  
    alert("Price is right! Buy the gadget.");  
} else {  
    alert("Price is too low or too high! Don't buy the gadget.");  
}
```

Bob's the better coder (and possibly, a better accountant, too).
Bob's solution works, but Bill's doesn't. To see why, let's try three different prices (too low, too high and just right) with Bob's and Bill's conditionals and see what results we get:

price	Bob's	Bill's
100	true alert: Don't buy!	true alert: Buy!
700	true alert: Don't buy!	true alert: Buy!
400	false alert: Buy!	true alert: Buy!

If price is 100, then 100 is less than 200, so Bob's conditional is true (remember, with OR, you only need one of the expressions to be true for the whole thing to be true), and we alert NOT to buy.

But Bill's conditional is also true, because price is ≤ 600 . So the result of the entire expression is true, and we alert the user to buy, even though the price is too low.

Turns out Bill's conditional is always true, no matter what the price is, so his code tells us to Buy! every time. Bill should stick with accounting.

EXERCISE SOLUTION

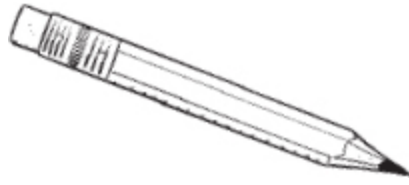


Remember we said pseudocode often isn't perfect? Well we actually left something out of our original pseudocode: we're not telling the user if her guess is a HIT or a MISS. Can you insert these pieces of code in the proper place to correct this? Here's our solution:

```
// Variable declarations go here
```

```
while (isSunk == false) {  
    guess = prompt("Ready, aim, fire! (enter a number from 0-6):");  
    if (guess < 0 || guess > 6) {  
        alert("Please enter a valid cell number!");  
    } else {  
        guesses = guesses + 1;  
        if (guess == location1 || guess == location2 || guess == location3) {  
            alert("HIT!");  
            hits = hits + 1;  
            if (hits == 3) {  
                isSunk = true;  
                alert("You sank my battleship!");  
            }  
        } else {  
            alert("MISS");  
        }  
    }  
}  
  
let stats = "You took " + guesses + " guesses to sink the battleship, " +  
            "which means your shooting accuracy was " + (3/guesses);  
alert(stats);
```

SHARPEN YOUR PENCIL SOLUTION



What do you think of this first attempt to write the code to detect when a ship is hit? Does it look more complex than it needs to be, or are we repeating code in a way that seems a bit, well, redundant? Could we simplify it? Using what you know of the `||` operator (that is, the boolean OR operator), can you simplify this code? *Here's our solution.*


```

if (guess == location1) {
    hits = hits + 1;
} else if (guess == location2) {
    hits = hits + 1;
} else if (guess == location3) {
    hits = hits + 1;
}

```

We're using the same code over and over here.

If we ever have to change how hits are updated, we've got three places to change our code. Changes like this are often a source of bugs and issues in code.

Not only that, this code is just way more complex than it needs to be. It's harder to read than it should be, and it took a lot more thought and typing than needed.

But, with the boolean OR operator we can combine the tests so that if guess matches any of location1, location2 or location3, then the if conditional will be true, and the hits variable will be updated.

```

if (guess == location1 || guess == location2 || guess == location3) {
    hits = hits + 1;
}

```

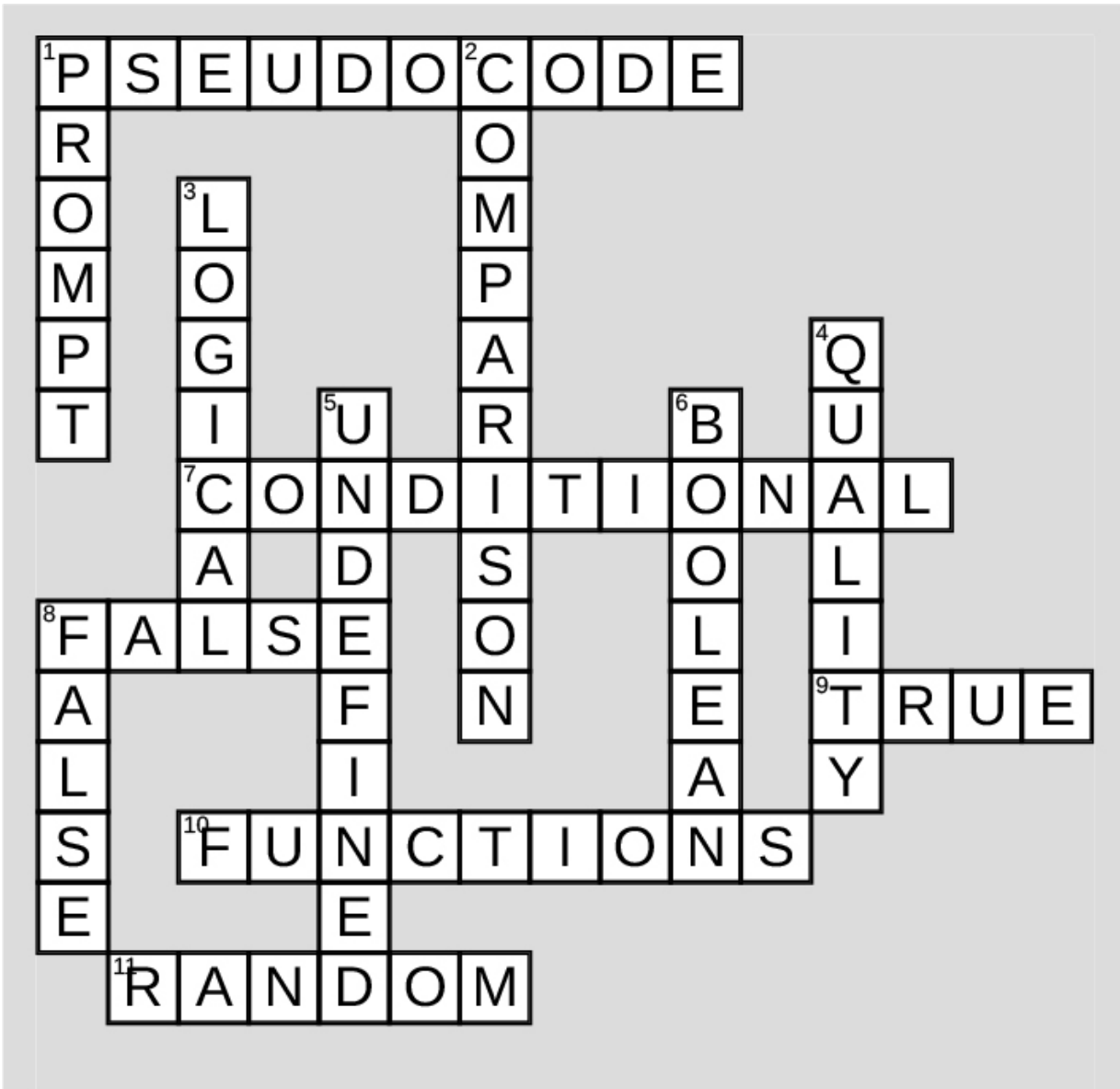
Isn't that much easier on the eye? Not to mention easier to understand.

And if we ever have to change how hits is updated, well, then we only have one place to do it, which is much less error prone.

JavaScript cross Solution



How does a crossword puzzle help you learn JavaScript? The mental twists and turns burn the JavaScript right into your brain! Here's our solution.



About the Authors

Eric Freeman is a computer scientist, technology writer, and entrepreneur. Previously, he was a CTO at the Walt Disney Company. Eric's most recent book, *Head First Learn to Code*, is a beginner's guide to coding and computational thinking. Eric lives with his wife and young daughter in Austin, Texas. He holds a PhD in computer science from Yale University.

Elisabeth Robson first got hooked on computers when she wrote a BASIC program to fill up her Commodore 64's monitor screen full of hearts. She's loved getting computers to do fun things ever since. She's tinkered with everything from super computers to her iPhone, and she's been programming the Web since the early days, when she built one of the first online resources for women in technology. Along with her many years of programming experience, Elisabeth has produced online training, and written four best-selling books and an online video course for O'Reilly Media (*Head First Design Patterns*, *Head First HTML and CSS*, *Head First JavaScript Programming*, *Head First HTML5 Programming*, and *Learn to Build iPhone Apps with HTML, CSS, and JavaScript*). She is currently co-founder and principal at WickedlySmart, an education content and technology company.